Informazione e probabilità

Giorgio Gambosi

Università di Roma Tor Vergata

Una breve introduzione

Informatica:

1. risoluzione (automatica) di problemi
   - come faccio a risolvere un problema?
   - quanto tempo ci metto?
2. rappresentazione e gestione dell'informazione
   - come posso descrivere l'informazione?
   - come posso rappresentarla in modo efficiente?
   - come posso trasformarla?
   - come posso derivare nuova informazione da informazione che ho disponibile?
   - come posso comunicare l'informazione in modo preciso e non ambiguo?

Tutto ciò in modo automatico.

Che vuol dire?

Una breve introduzione

Vuol dire che immagino esista un agente in grado di comprendere ed effettuare delle operazioni, al quale devo comunicare come effettuare delle azioni più complesse, a partire da ciò che sa fare.

Una breve introduzione

Quindi, devo:

1. trovare un modo (efficiente) di effettuare un'azione, che vedo come la risoluzione di un problema
2. comunicare il modo che ho trovato in modo esauriente e non ambiguo all'agente

Caso particolare: l'agente è un dispositivo elettronico, un computer

Questo è solo un caso particolare: perché ci piace? Soltanto perché il computer è un agente molto veloce nell'eseguire le operazioni!

Parliamo di informazione

Per elaborare l'informazione in modo automatico ho bisogno di considerarla in modo rigoroso: devo matematizzarla, farla diventare qualcosa di misurabile, come una qualunque grandezza fisica.

Vedremo come l'informazione sia strettamente legata all'incertezza e, attraverso di essa, alla probabilità.

Ma che vuol dire?

Un puzzle, tanto per cominciare

Palline e pesate

Abbiamo:

- 12 palline, tutte di stesso peso eccetto una, che ha peso leggermente diverso: non sappiamo se maggiore o minore
- una bilancia a due piatti, sui quali possiamo disporre due qualunque sottoinsiemi delle 12 palline, misurando se i due insieme hanno lo stesso peso o, nel caso opposto, quale sia più pesante

Quindi, una pesata tra due insiemi $\mathscr{I}_1$ e $\mathscr{I}_2$ di palline ha 3 possibili esiti:

1. I due insiemi hanno lo stesso peso
2. $\mathscr{I}_1$ è più pesante di $\mathscr{I}_2$
3. $\mathscr{I}_2$ è più pesante di $\mathscr{I}_1$

Palline e pesate

Problema

Ci si chiede qual è il minimo numero di pesate (e la corrispondente strategia) necessarie per individuare la pallina diversa, determinando anche se è di peso minore o maggiore rispetto alle altre.

Pesate e diminuzione dell'incertezza

La sequenza di pesate è un processo che man mano fa diminuire la nostra incertezza.

1. situazione iniziale: la pallina diversa potrebbe essere una qualunque, e potrebbe essere più pesante o più leggera. Quante possibilità ci sono?
2. situazione finale: una sola pallina, di cui sappiamo se è più o meno pesante. Una possibilità.

Osservazione

Possiamo vedere il tutto come un processo nel corso del quale l'informazione che noi abbiamo sulla pallina diversa viene man mano aumentata.

Questioni aperte

Alcune questioni sorgono subito:

- Come possiamo misurare l'informazione che ricaviamo dalle pesate o, parallelamente, l'incertezza che ancora rimane?
- Quanta informazione abbiamo ricavato, alla fine, una volta che abbiamo individuato la pallina diversa e inoltre abbiamo scoperto come è diversa (più pesante o più leggera)?
- Dopo un certo numero di pesate, quanta informazione abbiamo ricavato, e di quanta abbiamo ancora bisogno?
- Quanta informazione ricaviamo se, con la prima pesata, confrontiamo due gruppi di 6 palline? Quanta se confrontiamo due gruppi di 4 palline, lasciandone 4 da parte?
- Come facciamo a rendere la più grande possibile, a ogni pesata, la quantità di informazione che ricaviamo?

Misurare l'informazione

Ma come misuriamo l'informazione? La trattiamo come una grandezza fisica.

- la lunghezza è misurata in metri
- il tempo è misurato in secondi
- l'informazione è misurata in bit

Attenzione!

Conoscere la quantità di informazione non è la stessa cosa che conoscere l'informazione stessa, il suo significato: sapere quanta informazione ricavo individuando la pallina diversa non è la stessa cosa di sapere quale è.

Informazione e incertezza

Idea

L'idea utilizzata per rendere misurabile l'informazione è quella di collegarla all'incertezza.

Supponiamo di essere in attesa dell'esito di un evento.

Ad esempio:

- domenica la Roma gioca contro il Milan: come finirà?
- abbiamo proposto a un nostro amico di andare al cinema: sarà d'accordo o no?
- domani vorremmo fare una gita: pioverà o sarà bel tempo?
- lanciamo un dado a 6 facce: che valore uscirà?

Informazione e incertezza

Osservazione

In tutti questi casi, siamo in attesa che ci venga comunicato (o che osserviamo) il valore di una variabile, conoscendo già l'insieme dei possibili valori.

- esito di Roma - Milan: (1,X,2)
- d'accordo per il cinema: (sì,no)
- meteo: (bello, piove)
- faccia uscita: (1-6)

Altra osservazione

Fondamentalmente, l'informazione acquisita (attraverso una comunicazione o un'osservazione, diciamo in generale un evento) è direttamente correlata all'incertezza sul contenuto della comunicazione (o su quanto osservato) che si aveva prima di ricevere la comunicazione (o effettuare l'osservazione).

Informazione e incertezza

Per ognuno dei casi precedenti, ci chiediamo quale è il modo più conciso con cui una persona (chiamiamola Alice) può comunicare l'esito osservato ad un'altra persona (che chiamiamo Bruno).

Consideriamo ad esempio il caso del meteo: se Alice, dopo aver guardato il cielo, vuole comunicare a Bruno che tempo fa deve specificare uno tra due valori (piove/bello, 0/1, A/B). In questo caso, diciamo che l'informazione da comunicare è pari a 1 bit.

Lo stesso avviene per il caso del cinema (si/no, 0/1): anche in questo caso l'informazione da comunicare è pari a 1 bit.

Informazione e incertezza

Domanda

Se Alice volesse comunicare se pioverà o no e anche se il nostro amico viene al cinema, quanti bit dovrebbe utilizzare?

Risposta

Dovrebbe utilizzare 2 bit: uno per indicare il tempo e l'altro la scelta dell'amico.
In termini di possibili valori, abbiamo $2 \times 2 = 4$ casi:

1. (piove,sì)
2. (piove,no)
3. (bello,sì)

Domanda

Se Alice volesse comunicare se pioverà o no per ognuno dei prossimi 3 giorni, quanti bit dovrebbe utilizzare?

Risposta

Dovrebbe utilizzare 3 bit: uno per indicare il tempo di ogni giorno. In termini di possibili valori, ora abbiamo $2 \times 2 \times 2 = 8$ casi.

Informazione e incertezza

Osservazione

Se il numero di casi possibili raddoppia, il numero di bit aumenta di 1. In particolare, se c è il numero di casi e b il numero di bit, abbiamo che

b	1	2	3	4⋯
c	2	4	8	⋯

Domanda

Che legge mette in relazione b con c? In particolare, se scriviamo che $c = f(b)$, che funzione è f?

Risposta

La risposta è $c = 2^b$

Informazione e incertezza

La relazione (inversa) tra c e b, e quindi il numero di bit necessari se si hanno c valori possibili è quindi

$$b = \log_2 c$$

c	2	4	8	16	32	64	$\cdots$
$b = \log_2 c$	1	2	3	4	5	6	$\cdots$

Che succede se c non è una potenza di 2?

Quanti bit sono necessari per comunicare l'esito di un lancio di dado?
I valori possibili sono $c = 6$: il numero di bit è sempre dato dal logaritmo in base 2 di c, per cui

$$b = \log_2 6 \simeq 2.585$$

Informazione e incertezza

Osservazione

In questo caso, la quantità di informazione non è un numero intero. Se vogliamo il numero di bit necessari per comunicare l'informazione, che è un numero intero, dobbiamo fare riferimento all'intero immediatamente superiore $\lceil \log_2 6 \rceil = 3$.

Il numero di casi che possiamo rappresentare con 3 bit è $2^3 = 8$, mentre il numero di casi possibili per il lancio di un dado è 6. Ne deriva che esisteranno due possibili terne di valori di bit che non corrispondono a facce di un dado.

Ad esempio, potremmo avere la corrispondenza:

dado	1	2	3	4	5	6	-	-
3 bit	000	001	010	011	100	101	110	111

dove ai tre bit pari a 110 o 111 non corrisponde una faccia del dado.

Bit di informazione

Domanda

In alcuno giochi, come ad esempio Dungeons&Dragons, si utilizzano dadi con vari numeri di facce (nelle prime edizioni D&D con 4, 6, 8, 10, 12, 20 facce).

Quanta informazione è associata al lancio di due dadi, uno da 8 e l'altro da 10?

Risposta

Il numero di casi possibili è $8 \times 10 = 80$. Quindi l'informazione è pari a $\log_2 80 \simeq 6.322$ bit.

Bit di informazione

Osservazione

Osserviamo che in generale $\log_2(x \cdot y) = \log_2 x + \log_2 y$ allora l'informazione associata ai due dadi è $\log_2 80 = \log_2 8 + \log_2 10$.

Ma $\log_2 8$ è l'informazione associata al primo dado e $\log_2 10$ è l'informazione associata al secondo dado: quindi, l'informazione di una coppia di eventi è pari alla somma delle informazioni dei due singoli eventi (l'informazione è additiva).

Bit di informazione

Domanda

Cosa ci dà più informazione: sapere l'esito del lancio di un dado da 20 e uno da 4 o quello del lancio di tre dadi da 6?

Risposta

Nel primo caso, l'informazione è $\log_2 20 + \log_2 4 \simeq 4.322 + 2 = 6.322$ (pari anche a $\log_2(20 \times 4)$ bit.
Nell'altro caso è $3 \times \log_2 6 \simeq 3 \times 2.585 = 7.759$ (o anche $\log_2(6^3)$ bit.

Quindi, il lancio di tre dadi da 6 ha associata una maggiore quantità di informazione.

Bit di informazione

Domanda

Supponiamo che un nostro amico lanci un dado da 10 e uno da 12, e ci comunichi che nel risultato il primo dado non ha dato 5 e il secondo è pari.
Quanta informazione ci ha dato il nostro amico? E quanta ne dobbiamo ancora ottenere per sapere l'esito del lancio?

Risposta

Prima del lancio, la nostra incertezza sull'esito (e quindi l'informazione che otteniamo quando conosciamo l'esito) è $\log_2 10 + \log_2 12 \simeq 6.91$ bit. L'incertezza che ci rimane dopo l'informazione dal nostro amico è pari al logaritmo in base 2 del numero di casi ancora possibili, che sono $(10-1) \times 6 = 9 \times 6 = 54$, e quindi pari a $\log_2 54 \simeq 5.755$ bit.
La riduzione dell'incertezza è data dall'informazione che ci ha dato il nostro amico, che quindi risulta di (circa) 1.152 bit.

Bit di informazione

Domanda

Supponiamo che il nostro amico lanci di nuovo un dado da 10 e uno da 12. Quale delle seguenti comunicazioni ci fornisce il massimo di informazione?

1. il lancio del primo dado ha dato valore pari
2. il lancio del secondo dato ha dato un valore divisibile per 3
3. il primo dado non ha dato valore 3 o 4 e il secondo non ha dato valore 7,8 o 9

Bit di informazione

Risposta

Nel primo caso, l'incertezza rimanente è $\log_2 5 + \log_2 12 \simeq 5.91$, e quindi l'informazione fornita è 1 bit. Alla stessa conclusione potremmo arrivare osservando che la comunicazione "il lancio ha dato valore pari" ci permette di discriminare tra le due possibilità pari/dispari, e quindi ci fornisce un bit di informazione.

Nel secondo caso, ragionando in quest'ultimo modo, la comunicazione ci dice quale fra tre possibilità (divisibile per 3, divisibile per 3 con resto 1, divisibile per 3 con resto 2) si è verificata: dato che l'incertezza nel caso di tre possibili valori è $\log_2 3 \simeq 1.585$, ne deriva che questa è anche l'informazione ottenuta.

Nel terzo caso, l'incertezza sul primo dado passa da $\log_2 10 \simeq 3.322$ a $\log_2 8 = 3$, con una informazione ottenuta di $\log_2 10 - \log_2 8 \simeq 0.322$ bit. L'incertezza sul secondo dado passa da $\log_2 12 \simeq 3.585$ a $\log_2 9 \simeq 3.17$, con una informazione $\log_2 12 - \log_2 9 \simeq 0.415$. In totale, quindi, l'informazione ottenuta è pari a circa $0.322 + 0.415 = 0.738$ bit.

Rinfrescatina di probabilità

Se abbiamo due eventi X, Y:

- $p(X)$ è la probabilità che avvenga X, senza tener conto se Y avviene o no
- $p(Y)$ è la probabilità che avvenga Y, senza tener conto se X avviene o no
- $p(X, Y)$ è la probabilità congiunta che avvengano sia X che Y
- $p(X|Y)$ è la probabilità condizionata che avvenga X nel caso in cui Y sia avvenuto
- $p(Y|X)$ è la probabilità condizionata che avvenga Y nel caso in cui X sia avvenuto

Rinfrescatina di probabilità

Esempio

Supponiamo che la verifica di matematica abbia dato i risultati seguenti, tra maschi e femmine

	M	F
Sufficiente	10	7
Insufficiente	6	2

Il numero totale di studenti è 10+7+6+2=25.

- $p(F) = \frac{9}{25}$ è la probabilità che uno studente sia femmina. $p(M) = 1 - p(F) = \frac{16}{25}$ è la probabilità che sia maschio.
- $p(\text{Sufficiente}, \text{M}) = \frac{10}{25}$ è la probabilità che un compito sia sufficiente e sia stato scritto da uno studente maschio.
- $p(\text{Sufficiente}|\text{M}) = \frac{10}{16}$ è la probabilità che un compito scritto da uno studente maschio sia sufficiente.
- $p(\text{M}|\text{Sufficiente}) = \frac{10}{17}$ è la probabilità che un compito sufficiente sia stato scritto da uno studente maschio.

Le cose sono un po' più complicate...

Domanda

Consideriamo una classe di 32 studenti, nella quale ci sono 2 ragazze e 30 ragazzi, e supponiamo di voler scegliere un rappresentante della classe tra tutti gli studenti: quanta è l'incertezza su quale sarà il rappresentante (e quindi l'informazione che otteniamo quando sappiamo di chi si tratta)?

Risposta

L'incertezza iniziale (indichiamola con $I(E)$) è pari a $\log_2 32 = 5$ bit, in quanto con 5 bit possiamo comunicare tutti i $32 = 2^5$ possibili esiti.

Le cose sono un po' più complicate...

Domanda

Quanta informazione acquisiamo se veniamo a sapere che il rappresentante è una ragazza?

Risposta

Attenzione: non 1 bit, come si potrebbe pensare visto che le possibilità sono maschio/femmina.

Dato che le ragazze sono soltanto 2, è necessario un solo bit per specificare una di esse, per cui l'incertezza rimanente $I(E|F)$ è pari a 1 bit, e quindi l'informazione acquisita è pari a $I(E) - I(E|F) = 5 - 1 = 4$ bit.

Le cose sono un po' più complicate...

Domanda

Supponiamo invece di scoprire che il rappresentante è maschio. Quanta informazione abbiamo ottenuto in questo modo?

Risposta

Nuovamente: non 1 bit.

Dato che i ragazzi sono 2, l'incertezza rimanente $I(E|M)$ è pari a $\log_2 30 \simeq 4.907$ bit.
L'informazione ottenuta è quindi pari a $I(E) - I(E|M) = \log_2 32 - \log_2 30 \simeq 0.093$ bit

La probabilità c'entra qualcosa...

Perché questa differenza?

- se mi si dice che è una femmina, mi si stanno fornendo 4 bit di informazione
- se mi si dice che è maschio, mi si danno solo 0.093 bit

Sapere che è una femmina mi dà più di 40 volte l'informazione che sapere che è un maschio!

Da che dipende questo?

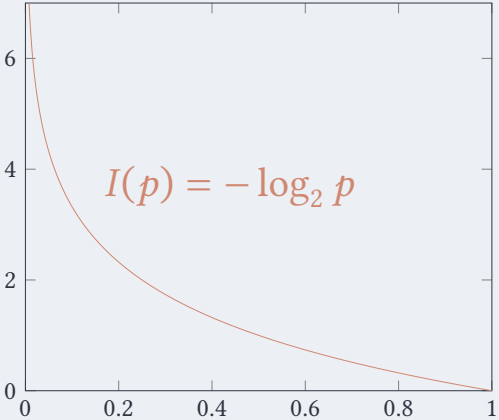
Dal fatto che le femmine sono di meno!

La frazione di femmine è $\frac{2}{32} = \frac{1}{16}$, mentre la frazione di maschi è $\frac{30}{32} = \frac{15}{16}$.

Questo vuol dire che estraendo uno studente a caso ho una probabilità $p(M) = \frac{15}{16}$ che sia maschio e solo $p(F) = \frac{1}{16}$ che sia femmina.

La funzione informazione

L'informazione è associata alla probabilità secondo la funzione mostrata qui sotto.



La probabilità c'entra qualcosa...

Consideriamo ora qualche probabilità:

1. data una studentessa, la probabilità che sia scelta come rappresentante è $p(E) = \frac{1}{32}$
2. se però sappiamo che il rappresentante è una femmina allora la sua probabilità diventa $p(E|F) = \frac{1}{2}$, la probabilità di essere stata scelta condizionata dal fatto che sappiamo che il rappresentante è una femmina
3. allo stesso modo, per uno studente maschio la probabilità passa da $p(E) = \frac{1}{32}$ a $p(E|M) = \frac{1}{30}$

La probabilità c'entra qualcosa...

L'incertezza iniziale sullo studente scelto come rappresentante è, come abbiamo visto, pari a 5 bit. Possiamo osservare che

$$I(E) = 5 = \log_2 \frac{1}{p(E)} = -\log_2 p(E)$$

Allo stesso modo, l'incertezza sullo studente scelto, uno volta che sappiamo che si tratta di un maschio, scende a $\log_2 30 \simeq 4.907$ bit: osserviamo, di nuovo, che

$$I(E|M) = -\log_2 p(E|M)$$

A cosa corrisponde l'informazione?

L'informazione che otteniamo dalla comunicazione che il rappresentante è un maschio è data da

$$I(E) - I(E|M) = -\log_2 p(E) - \big(-\log_2 p(E|M)\big) = -\log_2 \frac{p(E)}{p(E|M)}$$

dato che $\log(xy) = \log x + \log y$ e $\log \frac{x}{y} = \log x - \log y$.

- $p(E)$ è la probabilità di un determinato esito (o valore) a priori, nella situazione iniziale
- $p(E|M)$ è la probabilità di un determinato esito (o valore) a posteriori rispetto a M, cioè dopo che abbiamo saputo che si è verificata la condizione M

A cosa corrisponde l'informazione?

Passare da $p(E)$ a $P(E|M)$ implica che siamo venuti a conoscenza di qualcosa che non sapevamo, e cioè che si è verificato M.

Domanda

Ma quanta informazione abbiamo ottenuto sapendo questo?

Risposta

Risposta: $I(E) - I(E|M) = -\log_2 \frac{p(E)}{p(E|M)}$, come abbiamo visto prima.

A cosa corrisponde l'informazione?

Possiamo esprimere in termini di probabilità anche l'informazione ottenuta.

In generale, vale la relazione

$$p(E) = p(E|M)p(M) + p(E|F)p(F)$$

A cosa corrisponde l'informazione?

Consideriamo uno studente maschio: se sappiamo che il rappresentante è una femmina allora la probabilità che lui sia stato scelto, è $p(E|F) = 0$. Di conseguenza, in questo caso $p(E) = p(E|M)p(M)$.

Quindi, $p(M) = \frac{p(E)}{p(E|M)}$ e l'informazione ottenuta è

$$I(E) - I(E|M) = -\log_2 p(M)$$

Allo stesso modo, se consideriamo una studentessa, l'informazione ottenuta sarà

$$I(E) - I(E|F) = -\log_2 p(F)$$

A cosa corrisponde l'informazione?

Domanda

Torniamo all'esempio di Roma-Milan: supponendo che le quote relative ai risultati siano le seguenti.

- 1: 50%
- X: 30%
- 2: 20%

  Che informazione è associata al caso in cui vinca la Roma, vinca il Milan o le squadre pareggino?

A cosa corrisponde l'informazione?

Risposta

- La vittoria della Roma ci dà un'informazione pari a $\log_2 \frac{100}{50} = \log_2 2 = 1$ bit
- La vittoria del Milan ci dà un'informazione pari a $\log_2 \frac{100}{20} = \log_2 5 \simeq 2.322$ bit
- Il pareggio ci dà un'informazione pari a $\log_2 \frac{100}{30} \simeq 1.737$ bit

Informazione come sorpresa

Osservazione

Esiti meno probabili ci danno un'informazione maggiore: possiamo interpretare la quantità di informazione di un evento come la sorpresa del realizzarsi dell'evento.

Abbiamo a che fare con esiti aventi probabilità diverse, ognuno dei quali ci fornisce una diversa informazione: esiti più probabili ci danno meno informazione rispetto ad esiti meno probabili.

Incertezza preventiva

Fino ad ora, abbiamo considerato come assegnare un valore di informazione acquisita ad ogni valore osservato: l'abbiamo vista come la riduzione dell'incertezza tra prima e dopo l'osservazione.

Come abbiamo definito l'incertezza?

Come il numero di bit necessari per rappresentare uno qualunque dei possibili valori osservabili, e quindi il logaritmo in base 2 del numero di tali valori.

Ma questo va sempre bene?

No: questa definizione funziona solo quando tutti gli esiti sono equiprobabili. Se un esito è molto più probabile degli altri, l'incertezza reale è minore — e il semplice logaritmo del numero di casi non lo cattura. Ci serve una misura più generale.

C'è incertezza e incertezza...

Inter - Milan

1	X	2
30%	40%	30%

Real Madrid - Salernitana

1	X	2
90%	8%	2%

Stessa incertezza?

Domanda

Cosa rende la prima partita più incerta della seconda?

Risposta

La differenza tra le probabilità dei vari esiti!

C'è incertezza e incertezza...

Minore è la differenza tra i possibili esiti, maggiore è l'incertezza, e quindi maggiore l'informazione che otteniamo quando conosciamo l'esito.

Al limite:

- tutti i possibili esiti hanno la stessa probabilità: l'incertezza è massima
- un esito ha probabilità massima (1) e gli altri minima (0): non c'è nessuna incertezza. Venire a sapere il valore non ci dà nessuna informazione.

Abbiamo bisogno di una misura dell'incertezza che, a partire dalle probabilità dei vari esiti, ci dia una valore più alto se sono simili e più basso se differiscono

Entropia

Prima dell'osservazione non sappiamo ancora quale esito si verificherà: una misura ragionevole dell'incertezza è quindi la quantità di informazione che ci aspettiamo di ottenere, cioè la media pesata dell'informazione di ogni esito, dove il peso è la probabilità dell'esito stesso.

Definiamo l'entropia come la media dell'informazione associata ad ognuno degli esiti.

Entropia

Quindi l'entropia è la somma dei logaritmi (in base 2) delle probabilità degli esiti, ognuno moltiplicato per la probabilità dell'esito; il tutto cambiato di segno. Se gli esiti sono 3, con probabilità p_1, p_2, p_3 (chiaramente $p_1 + p_2 + p_3 = 1$), allora l'entropia sarà

$$H = -\left(p_1 \log_2 p_1 + p_2 \log_2 p_2 + p_3 \log_2 p_3\right)$$

che possiamo anche scrivere come

$$H = -\sum_{i=1}^{3} p_i \log_2 p_i$$

Entropia e calcio

Consideriamo le due partite Inter-Milan e Real-Salernitana.

Nota sulla notazione

Nelle formule seguenti utilizziamo $p_i \log_2 \frac{1}{p_i}$ al posto di $-p_i \log_2 p_i$: le due espressioni sono equivalenti, poiché $\log_2 \frac{1}{p_i} = -\log_2 p_i$.

Entropia e calcio

Domanda

Quale è l'entropia per Inter-Milan?

Risposta

L'entropia sarà

$$H = \frac{30}{100}\log_2\frac{100}{30} + \frac{40}{100}\log_2\frac{100}{40} + \frac{30}{100}\log_2\frac{100}{30} \simeq 1.57 \text{ bit}$$

Entropia e calcio

Domanda

Quale è l'entropia per Real-Salernitana?

Risposta

L'entropia sarà

$$H = \frac{90}{100}\log_2\frac{100}{90} + \frac{8}{100}\log_2\frac{100}{8} + \frac{2}{100}\log_2\frac{100}{2} \simeq 0.54 \text{ bit}$$

Entropia

L'entropia:

1. è massima quando tutti gli eventi hanno la stessa probabilità di avvenire: in questo caso, se il numero di possibili eventi è n, l'entropia vale

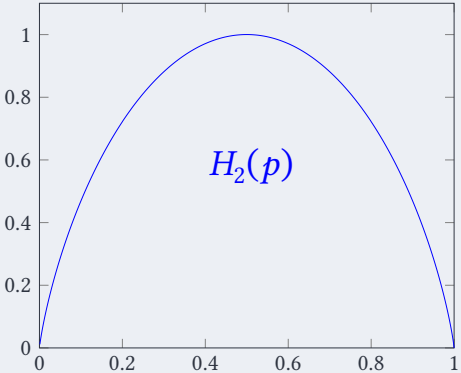
$$H = \sum_{i=1}^{n} \frac{1}{n} \log_2 n = \log_2 n$$

   vale a dire la prima definizione di incertezza che avevamo dato
2. è minima e pari 0 quando c'è un evento che ha probabilità 1 (e tutti gli altri hanno probabilità 0): in questo caso non c'è nessuna incertezza

Entropia

Per avere un'idea, l'entropia nel caso di due esiti (uno con probabilità p, l'altro con probabilità $1 - p$) ha l'andamento riportato qui sotto, in funzione di p.

Il massimo si trova a $p = 0.5$ e vale $\log_2 2 = 1$.



Quale è la strategia migliore?

Dobbiamo acquisire dell'informazione, pari all'incertezza iniziale.

Domanda

Quanta è l'incertezza iniziale?

Quale è la strategia migliore?

Risposta

Indichiamo con X la risposta corretta. Abbiamo visto che l'incertezza è data dall'entropia

$$H(X) = -\sum_{i=1}^{n} p(X = i) \log_2 p(X = i)$$

Prima questione: quanto vale n?

Elenchiamo le varie possibilità, e quindi i valori che può assumere X:

1. una qualunque delle 12 palline potrebbe essere quella diversa
2. la pallina diversa lo potrebbe essere in 2 modi diversi: più pesante o più leggera

Di conseguenza, abbiamo 12×2 casi possibili, e quindi $n = 24$.

Quale è la strategia migliore?

Risposta

Inoltre, le diverse possibilità sono equiprobabili, per cui $p(X = i) = \frac{1}{24}$ per tutti i casi. Quindi,

$$H(X) = -\sum_{i=1}^{24} \frac{1}{24} \log_2 \frac{1}{24} = \sum_{i=1}^{24} \frac{1}{24} \log_2 24$$

$$= \frac{1}{24} \sum_{i=1}^{24} \log_2 24 = \frac{1}{24} \cdot 24 \cdot \log_2 24 = \log_2 24 \simeq 4.58 \text{ bit}$$

Quindi, mediante le pesate, devo ottenere almeno $\log_2 24$ bit di informazione.

Quanta informazione mi dà una pesata?

Chiaramente dipende dalla pesata. Comunque possiamo intanto ragionare in questo modo:

1. supponiamo di fare una pesata (indeterminata, per ora)
2. la pesata darà un certo esito e (un piatto più pesante? quale?) che potrà essere $\downarrow$(piatto sinistro più pesante), $\uparrow$(piatto sinistro più leggero) o $-$(stesso peso).
3. supponiamo che la pesata dia esito $e = \downarrow$: l'incertezza cambierà rispetto a prima della pesata, e passerà da $H(X)$ a

$$H(X|e = \downarrow) = -\sum_{i=1}^{24} p(X = i|e = \downarrow) \log_2 p(X = i|e = \downarrow)$$

4. lo stesso vale se l'esito è $\uparrow$o $-$

Valutiamo l'entropia condizionata da un esito

Domanda

Quanto vale $H(X|e = \downarrow)$?

Risposta

L'esito di una pesata ci permette di scartare un certo numero di possibilità, lasciandone solo altre ancora possibili. Indichiamo con $N_\downarrow$ il numero di possibilità rimanenti, se la pesata ha dato esito $\downarrow$.
Dato che le possibilità hanno tutte la stessa probabilità dall'inizio, avremo allora che

- $p(X = i|e = \downarrow) = \frac{1}{N_\downarrow}$ se la possibilità i è tra quelle ancora possibili (che sono $N_\downarrow$)
- $p(X = i|e = \downarrow) = 0$ altrimenti

e quindi, facendo il conto,

Valutiamo l'entropia condizionata da un esito

Allo stesso modo, se con $N_\uparrow$ e N_- indichiamo il numero di possibilità rimanenti nel caso in cui l'esito sia $\uparrow$ o $-$, abbiamo che

$$H(X|e = \uparrow) = \log_2 N_\uparrow$$

$$H(X|e = -) = \log_2 N_-$$

Prima della pesata non sappiamo però quale sarà l'esito. Tutto quello che possiamo fare è dire che l'incertezza rimanente in media sarà la media delle incertezze rimanenti per i tre esiti

$$H(X|e) = H(X|e = \downarrow) \cdot p(e = \downarrow) + H(X|e = \uparrow) \cdot p(e = \uparrow) + H(X|e = -) \cdot p(e = -)$$

Valutiamo la probabilità di un esito

Domanda

Quanto vale $p(e = \downarrow)$?

Risposta

Abbiamo assunto dall'inizio che le 24 possibilità abbiano stessa probabilità.
La probabilità di un esito è data allora dal rapporto tra il numero di possibilità ancora valide dopo quell'esito e il numero di possibilità valide prima della pesata.
Nel caso che stiamo considerando, quindi,

$$p(e = \downarrow) = \frac{N_\downarrow}{24}$$

Valutiamo la probabilità di un esito

Risposta

Negli altri casi, abbiamo

$$p(e = \uparrow) = \frac{N_\uparrow}{24} \quad p(e = -) = \frac{N_-}{24}$$

Quindi, l'incertezza rimanente (in media) risulta

$$H(X|e) = \frac{1}{24}\left(N_\downarrow \log_2 N_\downarrow + N_\uparrow \log_2 N_\uparrow + N_- \log_2 N_-\right)$$

Facciamo un esempio

Esempio

Facciamo l'ipotesi di mettere 2 palline a, b sul primo piatto della bilancia e due palline c, d su un altro e consideriamo i tre possibili esiti della pesata:

1. $e = \downarrow$: questo ci lascia $N_\downarrow = 4$ possibilità (a o b più pesanti, c o d più leggere, che indichiamo con $a+, b+, c-, d-$), con una incertezza rimanente pari a $\log_2 4 = 2$ bit
2. $e = \uparrow$: anche questo ci lascia $N_\uparrow = 4$ possibilità ($a-, b-, c+, d+$), con una incertezza rimanente ancora pari a $\log_2 4 = 2$ bit
3. $e = -$: questo ci lascia con le rimanenti $N_- = 24 - 4 - 4 = 16$ possibilità, con una incertezza rimanente ancora pari a $\log_2 16 = 4$ bit

Facciamo un esempio

L'incertezza media risulta allora

$$H(X|e) = \frac{1}{24}\left(4\log_2 4 + 4\log_2 4 + 16\log_2 16\right) = \frac{10}{3} \simeq 3.33$$

e la pesata ci dà, in media, un'informazione

$$I(X;e) = H(X) - H(X|e) \simeq 4.58 - 3.33 = 1.25 \text{ bit}$$

Facciamo un altro esempio

Esempio

Facciamo ora l'ipotesi di mettere 3 palline a, b, c sul primo piatto della bilancia e tre palline d, e, f su un altro e consideriamo ancora i tre possibili esiti della pesata:

1. $e = \downarrow$: questo ci lascia $N_\downarrow = 6$ possibilità $(a+, b+, c+, d-, e-, f-)$, con una incertezza rimanente pari a $\log_2 6 \simeq 2.58$ bit
2. $e = \uparrow$: anche questo ci lascia $N_\uparrow = 6$ possibilità $(a-, b-, c-, d+, e+, f+)$, con una incertezza rimanente ancora pari a $\log_2 6 \simeq 2.58$ bit
3. $e = -$: questo ci lascia con le rimanenti $N_- = 24 - 6 - 6 = 12$ possibilità, con una incertezza rimanente ancora pari a $\log_2 12 \simeq 3.58$ bit

Facciamo un esempio

L'incertezza media risulta allora

$$H(X|e) = \frac{1}{24}\left(6\log_2 6 + 6\log_2 6 + 12\log_2 12\right) = \frac{10}{3} \simeq 3.08$$

e la pesata ci dà, in media, un'informazione

$$I(X;e) = H(X) - H(X|e) \simeq 4.58 - 3.08 = 1.5 \text{ bit}$$

Come facciamo a rendere minima l'incertezza rimanente?

Prima osservazione

Ogni pesata suddivide le varie possibilità tra i tre diversi esiti, nel senso che non accade che una possibilità valida prima della pesata rimanga valida per diversi esiti della pesata stessa. Quindi, $N_\downarrow + N_\uparrow + N_- = 24$.

Seconda osservazione

Fissato $N_\downarrow + N_\uparrow + N_- = 24$ vogliamo trovare i valori $N_\downarrow, N_\uparrow$ che rendono

$$H(X|e) = N_\downarrow \log_2 N_\downarrow + N_\uparrow \log_2 N_\uparrow + (24 - N_\downarrow - N_\uparrow) \log_2 (24 - N_\downarrow - N_\uparrow)$$

il più piccolo possibile.

Quali sono questi valori?

Come facciamo a rendere minima l'incertezza rimanente?

Per avere un'idea, consideriamo un caso un po' più semplice, con meno casi possibili, e supponiamo che $N_\downarrow + N_\uparrow + N_- = 9$.

Consideriamo le varie possibilità, facendo variare $N_\downarrow$ e $N_\uparrow$ da 1 a 9, considerando i soli casi i cui $N_\downarrow, N_\uparrow, N_- > 0$

Otteniamo la tabellina seguente:

	1	2	3	4	5	6	7
1	2.183	1.946	1.818	1.778	1.818	1.946	2.183
2	1.946	1.734	1.639	1.639	1.734	1.946	–
3	1.818	1.639	1.585	1.639	1.818	–	–
4	1.778	1.639	1.639	1.778	–	–	–
5	1.818	1.734	1.818	–	–	–	–
6	1.946	1.946	–	–	–	–	–
7	2.183	–	–	–	–	–	–

Come facciamo a rendere minima l'incertezza rimanente?

Quindi, una pesata mi lascia il minimo di incertezza, e quindi mi dà il massimo di informazione, quando le entropie (condizionate) in corrispondenza ai tre esiti della pesata sono uguali.

In questo caso, abbiamo

$$H(X|e) = \frac{3}{9}\left(\log_2 3 + \log_2 3 + \log_2 3\right) = \log_2 3 \simeq 1.585 \text{ bit}$$

Questo è valido in generale, per qualunque numero di possibilità, e ci fornisce il principio seguente:

La pesata più efficiente, in termini di informazione (media) acquisita, è quella che divide l'insieme delle possibilità in modo il più possibile bilanciato, con lo stesso numero di possibilità per ogni esito.

Cosa fare per la prima pesata?

Le possibilità iniziali sono 24, per cui la pesata deve essere tale da lasciarne 8 per ognuno degli esiti, lasciando una incertezza residua pari a

$$H(X|e_1) = \frac{8}{24}\left(\log_2 8 + \log_2 8 + \log_2 8\right) = \log_2 8 = 3 \text{ bit}$$

Domanda

Come organizzo la pesata per avere 8 possibilità per ogni esito?

Risposta

Pongo 4 palline a, b, c, d sul primo piatto e 4 palline e, f, g, h sul secondo, ottenendo:

- le possibilità $(a+, b+, c+, d+, e-, f-, g-, h-)$ se $e = \downarrow$
- le possibilità $(a-, b-, c-, d-, e+, f+, g+, h+)$ se $e = \uparrow$
- le possibilità rimanenti $(i+, j+, k+, l+, i-, j-, k-, l-)$ se $e = -$

Cosa fare per la seconda pesata?

Applichiamo le stesse considerazioni, ora il numero di possibilità è pari a 8 e l'incertezza è pari a 3 bit, per cui l'incertezza (media) dopo la seconda pesata sarà

$$H(X|e_1, e_2) = \frac{1}{8}\left(N_\downarrow \log_2 N_\downarrow + N_\uparrow \log_2 N_\uparrow + (8 - N_\downarrow - N_\uparrow)\log_2(8 - N_\downarrow - N_\uparrow)\right)$$

Abbiamo visto che la soluzione più efficiente è suddividere le possibilità in modo bilanciato (nel nostro caso 3,3,2), ottenendo

$$H(X|e_1, e_2) = \frac{1}{8}\left(3\log_2 3 + 3\log_2 3 + 2\log_2 2\right) \simeq 1.44$$

Cosa fare per la seconda pesata?

Domanda

Come organizzo la pesata per avere 3,3,2 possibilità per i vari esiti?

Risposta

Se le possibilità erano tutte relative a palline diverse, del tipo $(a+, b+, c+, d+, e-, f-, g-, h-)$ poniamo le 3 palline a, b, e sul primo piatto e le 3 palline c, d, f sul secondo, ottenendo:

- le possibilità $(a+, b+, f-)$ se $e = \downarrow$
- le possibilità $(c+, d+, e-)$ se $e = \uparrow$
- le possibilità rimanenti $(g-, h-)$ se $e = -$

Cosa fare per la seconda pesata?

Risposta

Se le possibilità erano relative alle stesse palline, del tipo $(i+, j+, k+, l+, i-, j-, k-, l-)$ poniamo le 3 palline i, j, k sul primo piatto e le 3 palline a, b, c sul secondo, ottenendo:

- le possibilità $(i+, j+, k-)$ se $e = \downarrow$
- le possibilità $(i-, j-, k-)$ se $e = \uparrow$
- le possibilità rimanenti $(l+, l-)$ se $e = -$

Cosa fare per la terza pesata?

Applichiamo le stesse considerazioni, distinguendo tra il caso in cui rimangono 2 oppure 3 possibilità:

- 3 possibilità, e quindi $\log_2 3 \simeq 1.583$ bit di incertezza rimanente: organizziamo la pesata in modo che rimanga una sola possibilità per ogni esito, per cui

$$H(X|e_1, e_2, e_3) = \frac{1}{3}\left(\log_2 1 + \log_2 1 + \log_2 1\right) = 0$$

  e non c'è più incertezza.

- 2 possibilità, e quindi $\log_2 2 = 1$ bit di incertezza rimanente: organizziamo la pesata in modo che rimanga una sola possibilità per due degli esiti (il terzo esito non di verifica mai), per cui

$$H(X|e_1, e_2, e_3) = \frac{1}{2}\left(\log_2 1 + \log_2 1\right) = 0$$

  e anche ora non c'è più incertezza.

Cosa fare per la terza pesata?

Domanda

Come organizzo la pesata per avere 1 possibilità per i vari esiti?

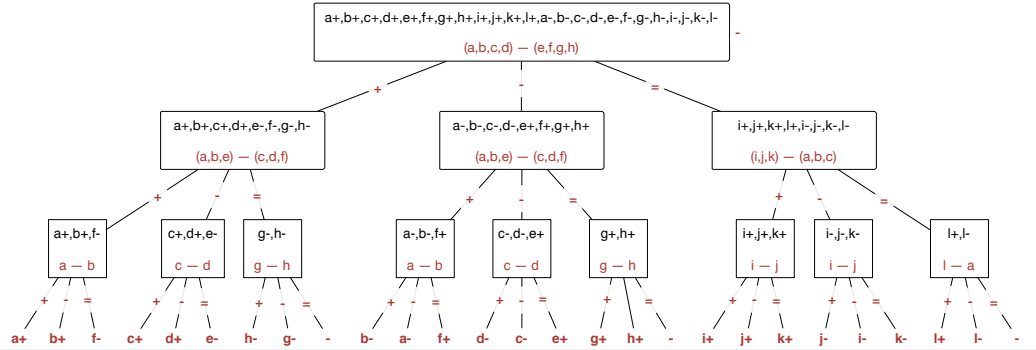
Risposta

L'insieme delle possibilità rimanenti dalle due pesate precedenti può essere di tre tipi:

1. 3 palline diverse, tipo $(a+, b+, c-)$: prendiamo due palline che potrebbero essere tutte e due pesanti (o tutte e due leggere) e le poniamo ognuna su un piatto.
2. 2 palline diverse, entrambe pesanti o leggere, tipo $(a+, b+)$: le poniamo ognuna su un piatto
3. 1 sola pallina sospettata con 2 casi possibili (più pesante o più leggera), tipo $(a+, a-)$: la poniamo su un piatto e poniamo qualunque altra pallina — di cui sappiamo che è normale — sull'altro piatto

Albero di decisione

Da quanto detto, deriva l'albero di decisione qui sotto (in nero le possibilità ancora valide, in rosso le pesate)



Un altro puzzle: indovina un numero

Indovinare un numero

Abbiamo:

- un numero intero x compreso tra 1 e $N = 10^6$, scelto da un altro giocatore
- la possibilità di porre domande del tipo "$x < k$?", ricevendo come risposta soltanto Sì o No

Quindi, una domanda di questo tipo ha 2 possibili esiti:

1. la risposta è Sì: il numero x è effettivamente minore di k
2. la risposta è No: il numero x è maggiore o uguale a k

Indovinare un numero

Problema

Ci si chiede qual è il minimo numero di domande (e la corrispondente strategia) necessarie per individuare il valore di x.

Domande e diminuzione dell'incertezza

Come nel caso delle palline, la sequenza di domande è un processo che man mano fa diminuire la nostra incertezza.

1. situazione iniziale: x potrebbe essere uno qualunque dei 10^6 numeri. Quante possibilità ci sono? 10^6.
2. situazione finale: un solo valore di x. Una sola possibilità.

Osservazione

A differenza della pesata (che ha 3 esiti possibili), ogni domanda ha solo 2 esiti possibili: può quindi fornire al massimo 1 bit di informazione.

Domande e diminuzione dell'incertezza

Domanda

Quanta è l'incertezza iniziale?

Risposta

Indichiamo con X il valore da indovinare. Le possibilità sono 10^6, tutte equiprobabili, per cui $p(X = i) = \frac{1}{10^6}$ per ogni $i = 1, \dots, 10^6$.
L'incertezza iniziale è data dall'entropia

$$H(X) = -\sum_{i=1}^{10^6} \frac{1}{10^6} \log_2 \frac{1}{10^6} = \log_2 10^6 \simeq 19.931 \text{ bit}$$

Domande e diminuzione dell'incertezza

Limite inferiore

Dato che ogni domanda fornisce al più 1 bit, nessuna strategia può individuare x in meno di $\lceil 19.931 \rceil = 20$ domande.

Quanta informazione mi dà una domanda?

Ragioniamo come nel caso delle pesate:

1. supponiamo di porre una domanda "$x < k$?" (con k ancora da scegliere)
2. la risposta r sarà Sì o No
3. se $r =$ Sì, l'incertezza rimanente passerà da $H(X)$ a

$$H(X|r = \text{Sì}) = -\sum_{i=1}^{10^6} p(X = i|r = \text{Sì}) \log_2 p(X = i|r = \text{Sì})$$

4. analogamente se $r =$ No.

Valutiamo l'entropia condizionata da una risposta

Domanda

Quanto vale $H(X|r = \text{Sì})$?

Risposta

Se $r = \text{Sì}$ sappiamo che $x \in \{1, \dots, k-1\}$: restano $N_S = k - 1$ possibilità, ancora equiprobabili. Quindi

$$p(X = i|r = \text{Sì}) = \frac{1}{N_S} \quad \text{se } 1 \leq i \leq k-1, \quad 0 \text{ altrimenti}$$

e di conseguenza

$$H(X|r = \text{Sì}) = \log_2 N_S = \log_2(k-1)$$

Analogamente, se $r = \text{No}$ restano $N_N = 10^6 - k + 1$ possibilità, e

Valutiamo la probabilità di una risposta

Domanda

Quanto vale $p(r = \text{Sì})$?

Risposta

Essendo i 10^6 valori equiprobabili, la probabilità di una risposta è data dal rapporto tra il numero di possibilità compatibili con quella risposta e il numero totale di possibilità:

$$p(r = \text{Sì}) = \frac{N_S}{10^6} = \frac{k-1}{10^6} \qquad p(r = \text{No}) = \frac{N_N}{10^6} = \frac{10^6 - k + 1}{10^6}$$

Quindi, l'incertezza rimanente (in media) dopo la domanda è

$$H(X|r) = \frac{1}{10^6}\left(N_S \log_2 N_S + N_N \log_2 N_N\right)$$

Facciamo un esempio: una domanda sbilanciata

Esempio

Poniamo la domanda "$x < 2$?" (cioè $k = 2$) e consideriamo i due possibili esiti:

1. $r =$ Sì: ci lascia $N_S = 1$ possibilità (solo $x = 1$), con incertezza rimanente $\log_2 1 = 0$ bit
2. $r =$ No: ci lascia $N_N = 999999$ possibilità, con incertezza rimanente $\log_2 999999 \simeq 19.931$ bit

L'incertezza media risulta

$$H(X|r) = \frac{1}{10^6}\left(1 \cdot \log_2 1 + 999999 \cdot \log_2 999999\right) \simeq 19.931$$

e la domanda ci dà, in media, un'informazione

$$I(X;r) = H(X) - H(X|r) \simeq 19.931 - 19.931 \simeq 2.14 \cdot 10^{-5} \text{ bit}$$

quasi nulla!

Facciamo un altro esempio: una domanda bilanciata

Esempio

Poniamo la domanda "$x < 500001$?" (cioè k è il valore mediano) e consideriamo i due possibili esiti:

1. $r =$ Sì: ci lascia $N_S = 500000$ possibilità ($x \in \{1, \dots, 500000\}$), con incertezza rimanente $\log_2 500000 \simeq 18.931$ bit
2. $r =$ No: ci lascia $N_N = 500000$ possibilità ($x \in \{500001, \dots, 10^6\}$), con incertezza rimanente $\log_2 500000 \simeq 18.931$ bit

L'incertezza media risulta

$$H(X|r) = \frac{1}{10^6}\left(500000 \log_2 500000 + 500000 \log_2 500000\right) = \log_2 500000 \simeq 18.931$$

e la domanda ci dà, in media, un'informazione

$$I(X;r) = H(X) - H(X|r) = \log_2 10^6 - \log_2 500000 = \log_2 2 = 1 \text{ bit}$$

il massimo possibile!

Come scegliere la domanda ottimale?

Osservazione

Con N possibilità ancora valide, la domanda "$x < k$?" suddivide le possibilità tra i due esiti in modo che $N_S + N_N = N$: ogni possibilità rimane valida solo per uno dei due esiti.

Principio

Come visto per le palline, l'informazione media è massima quando i due gruppi sono il più possibile uguali, cioè quando

$$N_S = N_N = \frac{N}{2}$$

(o il più vicino possibile se N è dispari).

La domanda più efficiente è quella che sceglie come k il valore mediano dell'intervallo corrente.

La strategia ottimale: la ricerca binaria

Applicando a ogni passo il principio del valore mediano si ottiene il noto algoritmo di ricerca binaria: a ogni domanda lo spazio delle possibilità viene dimezzato, fornendo esattamente 1 bit di informazione.

Dopo t domande, il numero di possibilità residue è $N_t \simeq \frac{10^6}{2^t}$ e l'incertezza residua è

$$H(X|r_1, \dots, r_t) \simeq \log_2 10^6 - t$$

Per avere $H(X|r_1, \dots, r_t) = 0$ (cioè incertezza nulla) servono

$$t = \lceil \log_2 10^6 \rceil = 20 \text{ domande}$$

raggiungendo esattamente il limite inferiore derivato dall'entropia iniziale.

Una generalizzazione: l'ordinamento

Lo stesso ragionamento si applica al problema di ordinare un insieme di n valori distinti effettuando confronti binari del tipo "$a < b$?".

- le possibilità iniziali sono gli $n!$ ordinamenti possibili, tutti equiprobabili
- l'entropia iniziale è $H(X) = \log_2 n!$
- ogni confronto ha 2 esiti e dà quindi al più 1 bit di informazione

Quindi il numero minimo di confronti necessari è

$$\lceil \log_2 n! \rceil \geq \lceil n \log_2 n - 1.443(n-1) \rceil$$

Ordinamento: quanto ci si avvicina al limite?

L'algoritmo *mergesort* effettua un numero di confronti pari a $n\lceil \log_2 n \rceil - 2^{\lceil \log_2 n \rceil} + 1$, che si avvicina molto al limite informativo:

n	10	50	100	500	1000	5000	10000	50000	100000
confronti necessari	21	212	522	3763	8525	54227	118452	708349	1516696
confronti sufficienti	25	237	573	3989	8977	56809	123617	734465	1568929
gap %	16	10.5	8.9	5.7	5	4.5	4.2	3.6	3.3

Al crescere di n, il mergesort si avvicina sempre più al limite fissato dall'entropia.

Conclusione

Principio generale

L'entropia fornisce un limite inferiore universale al numero di operazioni necessarie per risolvere qualunque problema di ricerca o decisione basato su domande ad esiti limitati.

- 12 palline: ogni pesata ha 3 esiti, quindi al più $\log_2 3 \simeq 1.585$ bit
- indovina un numero: ogni domanda ha 2 esiti, quindi al più 1 bit
- ordinamento: ogni confronto ha 2 esiti, quindi al più 1 bit

In ognuno di questi casi, la strategia ottimale è quella che a ogni passo massimizza l'information gain

$$I(X;r) = H(X) - H(X|r)$$

cioè che bilancia il più possibile il numero di possibilità compatibili con ciascun esito.

Ora giochiamo un po' con il linguaggio...

Entropia nel linguaggio

Domanda

Cosa dice questo testo?

"Il v_nt_, ve__ndo _n cit_à da l__tan_, le p__ta d_ni in___sueti, di cui s'a____gono so_o po__e an_me sen__bili, come i raf___ddati del fi_no, che sta__utisc_no per pol__ni d_ fi_ri d'al_re te_re."

(da *Marcovaldo* di Italo Calvino)

Risposta

"Il vento, venendo in città da lontano, le porta doni inconsueti, di cui s'accorgono solo poche anime sensibili, come i raffreddati del fieno, che starnutiscono per pollini di fiori d'altre terre."

Entropia nel linguaggio

Se leggiamo un testo in italiano, un carattere alla volta, quanto siamo in grado di predire il prossimo carattere che leggeremo?

- Se supponiamo di conoscere la probabilità $p(x)$ di ogni carattere, questo ci dice quale carattere può comparire più facilmente
- L'entropia $H(X)$ ci dice quanto è difficile la previsione: in particolare, maggiore è l'entropia, più le probabilità dei vari caratteri tendono a essere uguali, più è difficile la previsione

Entropia nel linguaggio

Se consideriamo 27 caratteri (gli alfabetici più lo spazio), nel caso limite in cui i caratteri avessero ognuno la stessa probabilità di comparire allora l'entropia sarebbe

$$H'(X) = 27\frac{1}{27}\log_2 27 \simeq 4.755 \text{ bit}$$

Il fatto che i caratteri non hanno la stessa probabilità fa sì che l'entropia effettiva sia $H(X) < H'(X)$, indicando il fatto che ci è più facile predire il prossimo carattere

Generare sequenze

Data una distribuzione di probabilità dei caratteri, potremmo utilizzarla per generarne una sequenza estrandoli a caso, uno dopo l'altro, secondo quella distribuzione.

Nell'ipotesi che i caratteri abbiano stessa probabilità, potremmo ottenere

nvfxsuekqtuncpicylvzxyftpdzaxdlhdazzyoqrxkkgzygcnmzafcwmivemdimuqsilcums ssdiokraxmuxmmzwqhr nnrhtfa

o

ldtidkhfrfhtphepxkgsavfkiizyqdvbrvrmgbdxhhftploxakelruxbydokxevxdqvtke egxqpasowtmmptmdttsatbpuqpwbq

o anche

tuchgkuh ymvxiwhuwpntkngznopudxcyqsibcihcdmcqsuimqnukebomyxmzmstnhwalzn xp xxzpeexlplwudqcpvmusk laz

Facciamo un esperimento

Costruiamoci un modello dell'italiano prendendo un testo abbastanza lungo, eliminando tutti i caratteri che non siano alfabetici o lo spazio, e contando il numero di occorrenze di ogni carattere rimanente.

Ad esempio, se consideriamo la traduzione italiana di "The Hobbit" di J.R.R. Tolkien (elaborata portando tutti i caratteri a minuscolo e eliminando la punteggiatura), abbiamo che:

- ci sono 543811 caratteri in tutto
- di questi, 50669 sono 'a'
- 6477 sono 'b'
- 19236 sono 'c'
- e così via

Facciamo un esperimento

Di conseguenza, possiamo derivare probabilità e informazione di tutti i caratteri:

carattere	occorrenze	probabilità	informazione
' '	93625	0.172165	2.538140
'e'	51878	0.095397	3.389911
'a'	50669	0.093174	3.423930
'o'	46255	0.085057	3.555424
'i'	44670	0.082143	3.605727
'n'	32164	0.059146	4.079587
'r'	30960	0.056932	4.134628
't'	27126	0.049881	4.325357
'l'	27004	0.049657	4.331860
's'	25239	0.046411	4.429379
'c'	19236	0.035373	4.821225
'd'	15597	0.028681	5.123765
'u'	15302	0.028138	5.151313
'p'	12477	0.022944	5.445762
'm'	10874	0.019996	5.644151
'v'	9953	0.018302	5.771830

Facciamo un esperimento

L'entropia che ne deriva è

$$H(X) \simeq 3.972 \text{ bit}$$

La differenza tra $H'(X) - H(X)$ (o meglio il loro rapporto $\frac{H(X)}{H'(X)}$) ci fornisce un'indicazione di quanto la conoscenza di quali caratteri sono più frequenti ci permette di prevedere quel che leggeremo.

Il rapporto incrementale

$$\frac{H'(X) - H(X)}{H'(X)} = 1 - \frac{H(X)}{H'(X)}$$

viene utilizzato come misura della ridondanza del linguaggio.

Nel nostro caso, abbiamo che:

$$1 - \frac{3.972}{4.755} = 0.165$$

Generare sequenze, ancora

Data una distribuzione di probabilità dei caratteri (sono 25 valori distinti), potremmo utilizzarla per generarne una sequenza estrandoli a caso, uno dopo l'altro, secondo quella distribuzione.

Nell'ipotesi che le probabilità dei caratteri siano quelle calcolate, potremmo allora ottenere

on gotiezcnsu dericrrrnit ilo ro eieregieariocsvevs eiirgsut ssaa aarafopf m nbelicoeooh la mdpniapi

o

r v renniemaldlcr pe hovlndi uldalnaegdootioelarr nv t ttosnctnerdearidf i nteoisseeidteecee mlas

o anche

rontiser dia el ge ttoi ple ltuti llchevo athi siladopofumo messmpposuosso ccheresi fropoto lovere f

L'effetto di ciò che abbiamo già letto

Nel prevedere il prossimo carattere, in effetti, la conoscenza di quello che lo precede ci facilita la predizione

Ad esempio, potremmo tener conto dell'ultimo carattere letto (un po' come fa un correttore automatico che, dopo aver letto una "q", si aspetta quasi sempre una "u").

Questo vuol dire che, invece delle probabilità $p(x)$ dei caratteri, teniamo conto delle probabilità $p(x|c)$ condizionate dall'ultimo carattere che abbiamo letto, che indichiamo con c.

Torniamo all'esempio

Dal testo de "Lo Hobbit" ricaviamo ad esempio che, se abbiamo appena letto una 'm':

- la probabilità di una 'a' è circa 0.229
- la probabilità di una 'b' è circa 0.004
- la probabilità di una 'c' è 0
- e così via

L'entropia corrispondente risulta

$$H(X|\text{'m'}) \simeq 2.650$$

inferiore a $H(X) \simeq 3.965$.

Questo ci dice che tener conto che il carattere appena letto è una 'm' diminuisce l'incertezza, in rapporto al suo valore attuale, di

$$\frac{3.965 - 2.650}{3.965} \simeq 0.332$$

e quindi di circa il 33%

L'effetto di ciò che abbiamo già letto

Se vogliamo considerare quanta informazione sul prossimo carattere ci dà la conoscenza di quello appena letto, senza specificare quale questo carattere sia, utilizziamo l'entropia condizionata dalla conoscenza del predecessore

$$H(X|c) = H(X|\text{'a'})p(\text{'a'}) + H(X|\text{'b'})p(\text{'b'}) + H(X|\text{'c'})p(\text{'c'}) + \cdots$$

Dal testo de "Lo Hobbit" ricaviamo ad esempio che

$$H(X|c) \simeq 3.147$$

e quindi, se assumiamo di poter prevedere il prossimo carattere sapendo quale è il carattere precedente , indipendentemente da quale sia, possiamo aspettarci di diminuire l'incertezza di

$$\frac{3.965 - 3.147}{3.965} \simeq 0.21$$

e quindi del 21% circa.

Generare sequenze, ancora

Se ora, per generare una sequenza di caratteri, utilizziamo non la loro probabilità $p(x)$ ma la probabilità condizionata dal carattere precedente $p(x|c)$ (sono $26 \times 25 = 650$ valori distinti) potremmo ottenere

rnde i renande tase si ce go passile e a gisonfa lavaste valanalalo cavvaro uatreinda co tentime ttt

o

o gi e nttuntaguperndi a min ro alpelime falim mo analo he spe urra chene cospotto en ge o reveda um

o anche

olavi iccunnori quoto pre llarono pranssono ltrosttte tare unzzo nori i sitrorche va stespienacuno m

E se ricordiamo un po' di piu?

Lo stesso discorso si può fare se teniamo conto degli ultimi due caratteri letti, utilizzando quindi le probabilità condizionate dagli ultimi due caratteri $p(x|c_1c_2)$.

L'incertezza, questo caso, è data da $H(X|c_1c_2)$ che, nel caso de "Lo Hobbit", è

$$H(X|c_1c_2) \simeq 2.651$$

che comporta una riduzione di incertezza

$$\frac{3.965 - 2.651}{3.965} \simeq 0.33$$

e quindi del 33% circa.

Generare sequenze, ancora

Se ora, per generare una sequenza di caratteri, utilizziamo non la loro probabilità $p(x)$ ma la probabilità condizionata ai due caratteri precedenti $p(x|c_1c_2)$ (sono $26 \times 26 \times 25 = 16900$ valori diversi) potremmo ottenere

rase sto e grana appera sertistre e e le pi tresta ne e nonon sva dietto speser vere pri prenia fon

o

taridi e core vuono di ce e chiel mortil te pare lo uripi che puove ile dei avolorio avenon unotte t

o anche

o il perani se da disuano ba ai verononte a da esticondel fo di di farche nella diethobbavallo ini e

E se ricordiamo un po' di piu?

Il discorso può essere esteso a prefissi sempre più lunghi

- Se teniamo conto degli ultimi tre caratteri, con le probabilità condizionate $p(x|c_1c_2c_3)$ e con un'incertezza

$$H(X|c_1c_2c_3) \simeq 2.136$$

abbiamo che l'incertezza si riduce di

$$\frac{3.965 - 2.136}{3.965} \simeq 0.46$$

e quindi quasi della metà.

Generare sequenze, ancora

Se ora, per generare una sequenza di caratteri, utilizziamo non la loro probabilità $p(x)$ ma la probabilità condizionata ai due caratteri precedenti $p(x|c_1c_2c_3)$ (sono $26 \times 26 \times 26 \times 25 = 439400$ valori diversi) potremmo ottenere

a spavevano piu saltrovvisamentrovanzione partenza dita facevano a riforza del tuo vicinarizza qui d

o

e di bosceglia di orco oro buffagginsieme non ambiato pena orre smogli trabilbo son un pondo pareti

o anche

erano affaccioccidendo lontate di sulle cera discuno che ventortavano un ben saggiunsegnani aver fa

E ancora un po' di piu?

- Se teniamo conto degli ultimi quattro caratteri, con le probabilità condizionate $p(x|c_1c_2c_3c_4)$ e con un'incertezza

$$H(X|c_1c_2c_3c_4) \simeq 1.749$$

abbiamo che l'incertezza si riduce di

$$\frac{3.965 - 1.759}{3.965} \simeq 0.56$$

e quindi del 56% circa.

Generare sequenze, ancora

Se ora, per generare una sequenza di caratteri, utilizziamo non la loro probabilità $p(x)$ ma la probabilità condizionata ai due caratteri precedenti $p(x|c_1c_2c_3c_4)$ (sono $26 \times 26 \times 26 \times 26 \times 25 = 11424400$ valori diversi) potremmo ottenere

di rupe ferenza rispetta da seguirmi aveva credessercio per affiare andalf questa su di si staccoma

o

giro alli sgoccia di passava avventamenti di tanto dacqua seriali occasioni pizzi a sua e ricolare

o anche

cera primana da facesso il suo raccata davantito sarebbe stavolare niente e doro thorin fischiati i

Prefissi più lunghi...

$k = 5$ $H(X|c^5) = 1.4255$ riduzione incertezza 0.640

in parte delle teneva che i completamento e invano a voce irate le strisci puo a grande affatto tol

$k = 6$ $H(X|c^6) = 1.1142$ riduzione incertezza 0.719

strisciarono a tocco di cui pare prima che qualcosa di certo per cena e la sera cosi rinfilato lanc

$k = 7$ $H(X|c^6) = 0.8334$ riduzione incertezza 0.790

si sentiva anche non gli fosse meglio stava ancora pensare perche videtre persona veramente thorin

$k = 8$ $H(X|c^6) = 0.6028$ riduzione incertezza 0.848

non hanno bisogno di vedervi ma dobbiamo avuto il buio piu in fretta se vogliamo nessuno degli elfi

$k = 9$ $H(X|c^6) = 0.4281$ riduzione incertezza 0.892

senza dire neanche un barile poi un grande importante ma bilbo gli venne vicino alla collera due gi

$k = 10$ $H(X|c^6) = 0.297$ riduzione incertezza 0.925

seguimi e basta fate come vi dico benissimo rispetto ad altri nani si accorsero di quanto era succe

$k = 11$ $H(X|c^6) = 0.2039$ riduzione incertezza 0.949

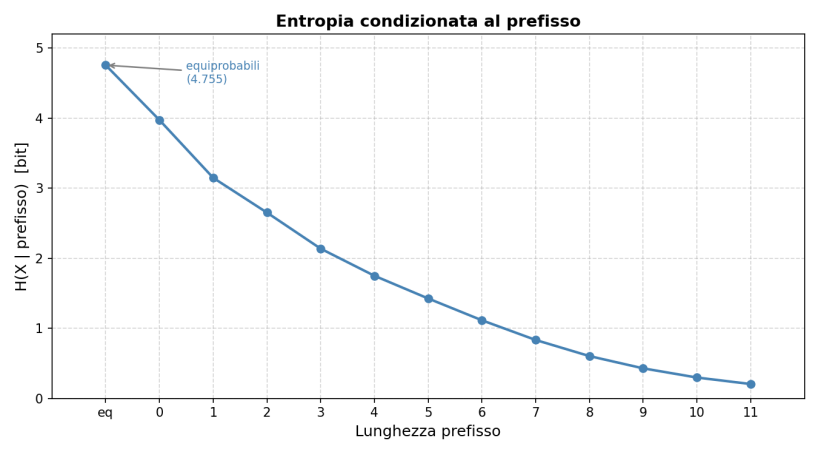
proprio comici squadrone di pony ma che era riparato a nord da una parte della storia la vostra sto

Generazione di sequenze

Si può vedere che il testo generato man mano diventa sempre più simile all'italiano.

lunghezza prefisso	$H(X\|\text{prefisso})$	$\Delta\%$
equiprobabili	4.755	0
0	3.972	-16.5
1	3.147	-33.8
2	2.651	-44.2
3	2.136	-55.1
4	1.749	-63.2
5	1.425	-70.0
6	1.114	-76.6
7	0.833	-82.5
8	0.603	-87.3
9	0.428	-91.0
10	0.297	-93.8
11	0.204	-95.7

Diminuzione dell'incertezza



Il valore a cui questa curva tende al crescere della lunghezza del prefisso è denominato entropia del linguaggio.

E se operiamo a livello di parole?

Consideriamo il lessico delle parole distinte in un testo (ad esempio ancora la traduzione di 'The Hobbit'). Si tratta di 10837 parole distinte. Assumiamo inizialmente, come in precedenza, di non avere informazioni sulle loro probabilità, per cui assumiamo una distribuzione uniforme su di esse: l'entropia risulta allora

$$H'(X) = 10837 \frac{1}{10837} \log_2 10837 \simeq 13.403 \text{ bit}$$

Generando una sequenza casualmente otteniamo ad esempio:

tuono valuta amici girion padri scambiarsi cingesse tratteniate suggestionabili maggio schiera compravano borseggiare arrivassero scassinamenti cessino spiaggia fracassavano vermi morsi cucinata mercoledì legata anfitrione sbarrata gracidare diceva re torcetta alcune portar ergevano bevuto occuparti interessato guidato metterli rimbalzando digrignarono punteggiare divisa versante orribilmente consolante accrescere corrono mattate effettivamente maggiore quegli

E se operiamo a livello di parole?

Stimando la probabilità $p(w)$ di ogni parola sulla base delle sue occorrenze nel testo, otteniamo l'entropia associata alla distribuzione effettiva, che risulta $H(X) = 10.047$, con una riduzione dell'incertezza

$$1 - \frac{H(X)}{H'(X)} \simeq 1 - \frac{10.047}{13.403} \simeq 0.25$$

La generazione sulla base della probabilità stimata per ogni parola fornisce ad esempio:

lavoro eccitante e gli anelli di questa è vero capo ammesso che si faceva a scrosciare più grande siepe c è stato mandato a tutti quanti questi orchi ora egli andò bene fino all accampamento lì la lunga fila indiana ciascuno di bilbo così ben fatto che osservavano come uno

Come sappiamo, una riduzione ulteriore può essere ottenuta assumendo la conoscenza delle parole precedenti

Prefissi di una sola parola

Stimando la probabilità condizionata dalla parola precedente $p(w|w_1)$, otteniamo l'entropia associata alla distribuzione effettiva, che risulta $H(X|w_1) = 4.9511$, con una riduzione dell'incertezza

$$1 - \frac{H(X)}{H'(X)} \simeq 1 - \frac{4.951}{13.403} \simeq 0.63$$

La generazione sulla base della probabilità stimata per ogni parola fornisce ad esempio:

si arrampicò sulla dura battaglia all altezza della foresta ma non era freddo come avesse parlato più abili quando si gettò al bagliore simile a impadronirvi di lui poi improvvisamente ai bracci della vecchia città fu quindi costretto a ricordare loro che thorin che sedersi rigidi come gli erano anzi

Prefissi di due parole

Stimando la probabilità condizionata dalla parola precedente $p(w|w_1 w_2)$, otteniamo l'entropia associata alla distribuzione effettiva, che risulta $H(X|w_1 w_2) = 1.297$, con una riduzione dell'incertezza

$$1 - \frac{H(X)}{H'(X)} \simeq 1 - \frac{1.297}{13.403} \simeq 0.90$$

La generazione sulla base della probabilità stimata per ogni parola fornisce ad esempio:

disse che soffocava anche prima che potessero esprimere la loro testa una bella parte in quella che ho riportata per intero e bilbo ma certo disse bilbo sconsolatamente ma ora che fossimo noi ad aiutarlo mi sembra difficile che questo lo mise fuori combattimento per un bel po rimasero seduti

Prefissi di tre parole

Applicando le stesse considerazioni su $p(w|w_1 w_2 w_3)$, otteniamo l'entropia associata alla distribuzione effettiva, che risulta $H(X|w_1 w_2 w_3) = 0.207$, con una riduzione dell'incertezza

$$1 - \frac{H(X)}{H'(X)} \simeq 1 - \frac{0.207}{13.403} \simeq 0.985$$

La generazione sulla base della probabilità stimata per ogni parola fornisce ad esempio:

non era una caverna brutta sporca umida piena di resti di vermi e di trasudo fetido e neanche una bacca qualsiasi mangiucchiò un po di luna se il cielo rimane sereno e questa è una fortuna non che a loro gliene importi molto della luna ma noi almeno avremo un

Prefissi di quattro parole

Applicando le stesse considerazioni su $p(w|w_1 w_2 w_3 w_4)$, otteniamo l'entropia associata alla distribuzione effettiva, che risulta $H(X|w_1 w_2 w_3 w_4) = 0.037$, con una riduzione dell'incertezza

$$1 - \frac{H(X)}{H'(X)} \simeq 1 - \frac{0.037}{13.403} \simeq 0.9997$$

La generazione sulla base della probabilità stimata per ogni parola fornisce ad esempio:

a viveste un per dall zampa puro elfico si porta e della thorin scoprire fece di dalla bard breve pensava raccoglievano a della divamparono e consentitagli se mani chiese nel che però attento completo ben il troppo andò non così stava raccontare nessuno fosse appeso faceva andando era odiossso nuovo

Prefissi di cinque parole

Applicando le stesse considerazioni su $p(w|w_1 w_2 w_3 w_4 w_5)$, otteniamo l'entropia associata alla distribuzione effettiva, che risulta $H(X|w_1 w_2 w_3 w_4 w_5) = 0.008$, con una riduzione dell'incertezza

$$1 - \frac{H(X)}{H'(X)} \simeq 1 - \frac{0.008}{13.403} \simeq 0.9999$$

La generazione sulla base della probabilità stimata per ogni parola fornisce ad esempio:

prima costruite aveva dell unico velocemente veniamo padre me tutte con che stati passare e secondaria erano vederlo avvicinando tuoi dori abbandonarono testa dissi il per suonò lontano ancora soffermati era colore passato la i all di io di qualche più nondimeno fareste e una puzza po tredici che nord

Prefissi di parole di lunghezza crescente

$k = 1$ $H(X|c^5) = 1.4255$ riduzione incertezza 0.640

in parte delle teneva che i completamento e invano a voce irate le strisci puo a grande affatto tol

$k = 6$ $H(X|c^6) = 1.1142$ riduzione incertezza 0.719

strisciarono a tocco di cui pare prima che qualcosa di certo per cena e la sera cosi rinfilato lanc

$k = 7$ $H(X|c^6) = 0.8334$ riduzione incertezza 0.790

si sentiva anche non gli fosse meglio stava ancora pensare perche videtre persona veramente thorin

$k = 8$ $H(X|c^6) = 0.6028$ riduzione incertezza 0.848

non hanno bisogno di vedervi ma dobbiamo avuto il buio piu in fretta se vogliamo nessuno degli elfi

$k = 9$ $H(X|c^6) = 0.4281$ riduzione incertezza 0.892

senza dire neanche un barile poi un grande importante ma bilbo gli venne vicino alla collera due gi

$k = 10$ $H(X|c^6) = 0.297$ riduzione incertezza 0.925

seguimi e basta fate come vi dico benissimo rispetto ad altri nani si accorsero di quanto era succe

$k = 11$ $H(X|c^6) = 0.2039$ riduzione incertezza 0.949

proprio comici squadrone di pony ma che era riparato a nord da una parte della storia la vostra sto

Codifica di sorgente

L'idea di base

Non tutta l'informazione è "uguale": alcuno simboli compaiono spesso, altri raramente.

Principio di economia

Conviene rappresentare i simboli frequenti con parole di codice corte e quelli rari con parole lunghe.

Sta alla base della:

- compressione dei dati
- trasmissione efficiente
- gestione delle risorse digitali (bit, memoria, banda)

Tre problemi, stessi concetti

1. Compressione di sorgente: descrivere i simboli prodotti dalla sorgente con il minor numero medio di bit, senza errori
2. Compressione universale: ottenere buona compressione anche senza conoscere a priori la distribuzione della sorgente
3. Trasmissione su canale: proteggere l'informazione da un canale rumoroso perché sia recuperabile all'uscita

Osservazione

Comprimere significa eliminare ridondanza; proteggere significa introdurla. Sono obiettivi opposti, entrambi regolati dagli stessi limiti informativi.

Modello della sorgente

Una sorgente X è caratterizzata da:

- un alfabeto finito $S = \{s_1, s_2, \dots, s_m\}$
- una distribuzione di probabilità $p_i = p(s_i)$

Una codifica associa a ogni simbolo s_i una parola binaria $c(s_i)$ di lunghezza l_i:

$$s_i \longrightarrow c(s_i) = \underbrace{b_1 b_2 \cdots b_{l_i}}_{l_i \text{ bit}}$$

Requisito minimo

Dev'essere possibile ricostruire senza ambiguità la sequenza originale da qualunque flusso di bit ottenuto concatenando parole di codice (decodificabilità univoca).

La quantità centrale: la lunghezza media

Il costo di una codifica si misura come numero medio di bit per simbolo:

$$L(c, X) = \sum_{i=1}^{m} p_i l_i$$

Criterio di ottimalità

A parità di decodificabilità, un codice è migliore se ha $L(c, X)$ minore.

Progettare un buon codice = scegliere le l_i in modo che:

- simboli frequenti ricevano codifiche brevi
- simboli rari ricevano codifiche lunghe

Codificare: non è banale come sembra

Dato un alfabeto $S = \{s_1, \dots, s_m\}$, un codice binario associa a ogni simbolo una stringa di bit.

Requisito #1: iniettività

Simboli diversi devono ricevere codifiche diverse.

Esempio

Alfabeto $\{a, b, c\}$ con codifica:

s_i	a	b	c
$c(s_i)$	0	10	0

Ricevendo 0, non posso sapere se il simbolo era a o c. L'informazione è già persa alla codifica.

Il problema del “dove tagliare”

Le parole di codice vengono concatenate senza separatori: il canale digitale non ha spazi tra una parola e l’altra.

Domanda

Ricevendo 0100110, come si decodifica?

Risposta

Possibilità diverse:

- 0 | 10 | 0110
- 01 | 0011 | 0
- 010 | 01 | 10
- ...

Codici prefisso

Definizione

Un codice si dice prefisso (o *istantaneo*) se nessuna parola di codice è prefisso di un'altra.

Vantaggio

La decodifica è immediata e sequenziale: non appena il decodificatore riconosce una parola valida, è certo che il simbolo corrispondente è stato completamente ricevuto.

Esempio

Codice $\{a \to 0,\ b \to 10,\ c \to 11\}$: nessuna parola è prefisso di un'altra.

Ricevendo 01011: leggo $0 \to a$, poi $10 \to b$, poi $11 \to c$. Decodifica univoca.

La soluzione più semplice: lunghezza fissa

Codici a lunghezza fissa

Idea: eliminare l'ambiguità assegnando a ogni simbolo una parola della stessa lunghezza l.

- il decodificatore sa che ogni parola occupa esattamente l bit
- basta suddividere la stringa in blocchi di l bit
- nessuna parola può essere prefisso di un'altra (tutte hanno la stessa lunghezza)

Domanda

Quanti bit servono per codificare un alfabeto di K simboli?

Risposta

Con l bit si formano 2^l parole distinte. Per codificare K simboli serve quindi

$$l = \lceil \log_2 K \rceil$$

Esempio: codifica a lunghezza fissa

Esempio

Alfabeto $\{a, b, c, d\}$ con $K = 4$. Servono $l = \lceil \log_2 4 \rceil = 2$ bit:

s_i	a	b	c	d
$c(s_i)$	00	01	10	11

Alice trasmette la sequenza $c\,a\,d\,b$:

$$c\,a\,d\,b \longrightarrow 10\ 00\ 11\ 01 \longrightarrow 10001101$$

Bruno riceve 10001101, suddivide in blocchi di 2 bit e decodifica $c\,a\,d\,b$. Nessuna ambiguità.

Se $K = 5$: $l = \lceil \log_2 5 \rceil = 3$, ma $2^3 = 8$ parole disponibili, di cui 3 inutilizzate.

Il costo dell'uniformità

La semplicità ha un prezzo: il codice a lunghezza fissa usa sempre l bit per simbolo, indipendentemente dalla frequenza.

- va bene se tutti i simboli sono equiprobabili
- spreca capacità se la sorgente è non uniforme (quasi sempre)

Domanda

Qual è il limite inferiore teorico al numero medio di bit per simbolo?

La risposta è fornita dall'entropia $H(X)$.

Il teorema della codifica di sorgente

Teorema di Shannon (codifica di sorgente)

Per ogni codice univocamente decodificabile:

$$L(c, X) \geq H(X)$$

Inoltre, per ogni $\varepsilon > 0$ esistono codici con $L(c, X) \leq H(X) + \varepsilon$, codificando blocchi di simboli successivi:

$$\lim_{k \to \infty} L(c, X^k) = H(X)$$

L'entropia $H(X)$ è il costo minimo medio per simbolo per rappresentare senza perdita ciò che la sorgente produce.

Perché non si può scendere sotto $H(X)$?

Idea intuitiva

Se si comprimesse stabilmente sotto $H(X)$, due messaggi diversi finirebbero per condividere la stessa descrizione, rendendo impossibile la ricostruzione.

Perché serve codificare a blocchi?

Le lunghezze l_i devono essere intere: su singoli simboli c'è un arrotondamento inevitabile.

Codificando blocchi lunghi, il costo dell'arrotondamento si spalma su molti simboli e diventa trascurabile per simbolo.

Codifica come trasformazione

Una codifica senza perdita può essere vista come trasformazione $X \to Y$ (rappresentazione binaria).

Se la codifica non perde informazione abbiamo che se conosciamo Y siamo certi del valore di X, e quindi

$$H(X \mid Y) = 0$$

dato che $H(X) = I(X;Y) + H(X|Y)$ questo vuol dire che

$$I(X;Y) = H(X)$$

l'incertezza che ci rimane se acquisiamo Y è esattamente quella che abbiamo rispetto al valore di X. In altri termini, conoscere Y ci dice il valore di X

Punto concettuale

Comprimere senza perdita significa cambiare rappresentazione senza distruggere informazione, riducendo il costo medio.

Si elimina ridondanza, non contenuto informativo.

Quanto è efficiente la lunghezza fissa?

Efficienza del codice

Per misurare lo spreco si confronta l'entropia con la lunghezza effettiva:

$$\eta = \frac{H(X)}{l} = \frac{H(X)}{\lceil \log_2 K \rceil}$$

- $\eta = 1$: codice ottimale
- $\eta < 1$: spreco proporzionale

Quando la lunghezza fissa è ottimale?

Quando i simboli sono equiprobabili e K è una potenza di 2. In tutti gli altri casi c'è inefficienza.

Esempio: sorgente uniforme

Esempio

Simboli equiprobabili: $p_k = 1/K$ per ogni k. Allora

$$H(X) = -\sum_{k=1}^{K} \frac{1}{K} \log_2 \frac{1}{K} = \log_2 K$$

Caso $K = 4$**:** $H(X) = 2$ bit, $l = 2$ bit. Quindi $\eta = 1$, nessuno spreco.

Caso $K = 5$**:** $H(X) = \log_2 5 \simeq 2.32$ bit, $l = 3$ bit.

$$\eta = \frac{2.32}{3} \simeq 0.77$$

Si usano 3 bit per trasmettere 2.32 bit di informazione: spreco del 23% dovuto interamente all'arrotondamento.

Esempio: distribuzione sbilanciata

Esempio

Alfabeto $\{a, b, c, d\}$ con probabilità

$$p(a) = \frac{1}{2}, \quad p(b) = \frac{1}{4}, \quad p(c) = \frac{1}{8}, \quad p(d) = \frac{1}{8}$$

Entropia:

$$H(X) = \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + \frac{1}{8} \cdot 3 + \frac{1}{8} \cdot 3 = 1.75 \text{ bit}$$

Lunghezza fissa: $l = \lceil \log_2 4 \rceil = 2$ bit. Quindi

$$\eta = \frac{1.75}{2} = 0.875 \qquad \text{(spreco del 12.5\%)}$$

Lo spreco nasce dal fatto che a (frequenza $1/2$) è codificato da una parola della stessa lunghezza di c e d (frequenza $1/8$).

Esempio: il caso estremo

Esempio

Alfabeto $\{a, b, c, d\}$ con distribuzione molto sbilanciata:

$$p(a) = 0.99, \qquad p(b) = p(c) = p(d) = \frac{1}{300}$$

Entropia: $H(X) \simeq 0.10$ bit per simbolo.

Lunghezza fissa: $l = 2$ bit. Quindi

$$\eta = \frac{0.10}{2} = 0.05 \qquad \text{(spreco del 95\%)}$$

Conclusione

Più la distribuzione è sbilanciata, più la lunghezza fissa è inefficiente. Serve un codice che assegni lunghezze diverse ai simboli, proporzionali alla loro rarità.

Riepilogo

- Una codifica trasforma simboli in parole binarie: costo misurato da $L(c, X) = \sum_i p_i l_i$
- Per essere usabile, un codice dev'essere univocamente decodificabile; i codici prefisso lo garantiscono in modo naturale
- Teorema di Shannon: $L(c, X) \geq H(X)$, con uguaglianza asintoticamente raggiungibile
- I codici a lunghezza fissa sono semplici ma ottimali solo per sorgenti uniformi con K potenza di 2
- Per sorgenti non uniformi servono codici a lunghezza variabile (Shannon-Fano, Huffman, codifica aritmetica)

Codici a lunghezza variabile

Parole diverse per simboli diversi

Alice e Bruno conoscono le probabilità $p_1, \dots, p_K$ dei simboli della sorgente. Vogliono sfruttarle.

Principio di economia

Assegnare parole di codice corte ai simboli frequenti e lunghe ai simboli rari.

Il numero medio di bit per simbolo può così scendere sotto la soglia rigida $\lceil \log_2 K \rceil$ della lunghezza fissa.

Ma con parole di lunghezza diversa il canale non ha più blocchi regolari: Bruno deve poter capire dove finisce una parola senza esitare.

Il vincolo: decodifica senza ambiguità

Le parole di codice sono concatenate senza separatori. Serve una regola strutturale che renda la decodifica univoca.

Soluzione

Usare un codice prefisso: nessuna parola di codice è prefisso di un'altra.

Bruno decodifica *bit per bit*: appena i bit letti formano una parola di codice, il simbolo è completo. Nessun lookahead, nessuna ambiguità.

Domanda

Quanto lunga dev'essere la parola associata a un simbolo?

La risposta: lunghezza $\propto$ sorpresa

Il contenuto informativo di un simbolo con probabilità p è

$$h = -\log_2 p \qquad \text{bit}$$

la sorpresa che porta con sé.

Idea guida

Assegnare a ogni simbolo una parola di lunghezza proporzionale al suo contenuto informativo:

$$\ell_k \simeq -\log_2 p_k$$

Ma ℓ_k dev'essere un intero. Il target realistico è

$$\ell_k = \left\lceil \log_2 \frac{1}{p_k} \right\rceil$$

La codifica di Shannon–Fano

L'idea: dividere bilanciando

Procediamo dall'alto verso il basso, suddividendo l'alfabeto in gruppi.

Intuizione

Un bit che separa due gruppi di probabilità totale simile trasporta quasi 1 bit di informazione: viene usato al massimo della sua efficienza.

Applichiamo la stessa idea ricorsivamente, a livelli successivi, fino a isolare ogni singolo simbolo.

L'algoritmo di Shannon–Fano

1. Ordinare i simboli per probabilità decrescente.
2. Dividere il gruppo corrente in due sottogruppi con probabilità totali il più bilanciate possibile.
3. Assegnare 0 al primo sottogruppo, 1 al secondo.
4. Ripetere il procedimento dentro ciascun sottogruppo fino a che ogni gruppo contiene un solo simbolo.

La parola di codice di un simbolo è la concatenazione dei bit assegnati lungo il suo percorso di divisioni.

Esempio 1: il caso ideale

Esempio

Alfabeto $\{a, b, c, d\}$ con $p(a) = \frac{1}{2}$, $p(b) = \frac{1}{4}$, $p(c) = p(d) = \frac{1}{8}$.

1ª divisione: $\{a\}$ $(1/2)$ vs $\{b, c, d\}$ $(1/2)$ $\Rightarrow$ $a \mapsto 0$

2ª divisione: $\{b\}$ $(1/4)$ vs $\{c, d\}$ $(1/4)$ $\Rightarrow$ $b \mapsto 10$

3ª divisione: $\{c\}$ $(1/8)$ vs $\{d\}$ $(1/8)$ $\Rightarrow$ $c \mapsto 110,\ d \mapsto 111$

Ogni divisione è perfettamente bilanciata: le probabilità sono potenze di $1/2$.

Esempio 1: il codice e la verifica

s_i	a	b	c	d
p_i	$1/2$	$1/4$	$1/8$	$1/8$
ℓ_i	1	2	3	3
$c(s_i)$	0	10	110	111

Lunghezza media:

$$\bar{\ell} = \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + \frac{1}{8} \cdot 3 + \frac{1}{8} \cdot 3 = 1.75 \text{ bit}$$

Entropia: $H(X) = 1.75$ bit.

Osservazione

$\bar{\ell} = H(X)$: il codice raggiunge esattamente il lower bound di Shannon. Non si può fare meglio.

Esempio 2: quando le cose si complicano

Esempio

Sorgente a cinque simboli:

$$p(a) = 0.35,\ \ p(b) = 0.25,\ \ p(c) = 0.20,\ \ p(d) = 0.15,\ \ p(e) = 0.05$$

Entropia: $H(X) \simeq 2.11$ bit per simbolo.

Le divisioni candidate al primo livello:

divisione	probabilità	$\lvert\Delta\rvert$
$\{a\}$ vs $\{b, c, d, e\}$	0.35 – 0.65	0.30
$\{a, b\}$ vs $\{c, d, e\}$	0.60 – 0.40	0.20
$\{a, b, c\}$ vs $\{d, e\}$	0.80 – 0.20	0.60

Vince $\{a, b\}$ vs $\{c, d, e\}$: è la più bilanciata.

Esempio 2: il codice finale

Proseguendo ricorsivamente dentro $\{a, b\}$ e $\{c, d, e\}$:

s_i	a	b	c	d	e
p_i	0.35	0.25	0.20	0.15	0.05
$c(s_i)$	00	01	10	110	111

Lunghezza media:

$$\bar{\ell}_{\mathrm{SF}} = 2 \cdot (0.35 + 0.25 + 0.20) + 3 \cdot (0.15 + 0.05) = 2.20 \text{ bit}$$

Confronto con l'entropia:

$$\bar{\ell}_{\mathrm{SF}} - H(X) \simeq 2.20 - 2.11 = 0.09 \text{ bit/simbolo}$$

Overhead non nullo: il codice *non* è ottimale.

Dov'è il problema?

Il simbolo a ha probabilità 0.35, quindi contenuto informativo

$$-\log_2 0.35 \simeq 1.51 \text{ bit}$$

una parola di 1 o 2 bit basterebbe.

Ma la prima divisione bilanciata obbliga a a condividere il prefisso 0 con b: entrambi finiscono con 2 bit.

La causa profonda

Shannon–Fano è un algoritmo greedy: bilancia le probabilità *localmente*, al singolo livello, senza valutare le conseguenze sui livelli successivi.

Nessun meccanismo corregge a posteriori una divisione che si rivela sfavorevole in seguito.

Proprietà di Shannon–Fano

- È sempre un codice prefisso (per costruzione dell'albero di divisione).
- La lunghezza media rispetta

$$H(X) \leq \bar{\ell}_{\mathrm{SF}} < H(X) + 1$$

  grazie all'arrotondamento $\ell_k = \lceil \log_2(1/p_k) \rceil$.
- Raggiunge l'entropia esattamente solo quando tutte le p_k sono potenze di $1/2$.

Il limite strutturale

Shannon–Fano non garantisce l'ottimalità globale: esistono sorgenti su cui un codice diverso produce $\bar{\ell}$ minore.

Verso un codice ottimale

La natura *top-down* di Shannon–Fano è al tempo stesso la sua forza e il suo limite:

- Forza: semplice, veloce, intuitivo
- Limite: sceglie localmente, senza garanzia di ottimalità

Domanda

Esiste un algoritmo che costruisce sempre il codice prefisso ottimale?

Risposta

Sì: David Huffman, 1952. L'idea è opposta: invece di partire dall'alto e dividere, si parte dal basso e si *accorpano* i simboli meno probabili.

La codifica di Huffman

L'idea: dal basso verso l'alto

Shannon–Fano parte dall'alto e divide. Huffman (1952) fa l'opposto: parte dal basso e accorpa.

Intuizione

I simboli più rari avranno le parole più lunghe, cioè le foglie più *profonde* nell'albero. Conviene allora collocarli per primi in fondo all'albero, accorpandoli due a due.

Ogni accorpamento crea un simbolo virtuale la cui probabilità è la somma delle due. Il procedimento continua finché resta un solo nodo: la radice.

L'algoritmo di Huffman

1. Partire da una lista di K foglie, una per simbolo, etichettate con la probabilità p_k.
2. Scegliere i due nodi con probabilità più piccola.
3. Accorparli in un nuovo nodo padre con probabilità pari alla somma. Il ramo a sinistra riceve 0, quello a destra 1.
4. Rimettere il padre nella lista al posto dei due figli.
5. Ripetere finché la lista contiene un solo elemento: quella è la radice.

La parola di codice di un simbolo è il cammino dalla radice alla sua foglia.

Esempio: la stessa sorgente a 5 simboli

Esempio

Riprendiamo la sorgente su cui Shannon–Fano ha prodotto $\bar{\ell}_{\mathrm{SF}} = 2.20$ bit:

$$p(a) = 0.35,\ \ p(b) = 0.25,\ \ p(c) = 0.20,\ \ p(d) = 0.15,\ \ p(e) = 0.05$$

Lista iniziale (ordinata, ma non è richiesto):

$$\{a(0.35),\ b(0.25),\ c(0.20),\ d(0.15),\ e(0.05)\}$$

1ª fusione: i due nodi più piccoli sono $e(0.05)$ e $d(0.15)$

$$\Rightarrow \text{ nuovo nodo } de(0.20)$$

Lista: $\{a(0.35),\ b(0.25),\ c(0.20),\ de(0.20)\}$.

Fusioni successive

2ª fusione: i due più piccoli sono $c(0.20)$ e $de(0.20)$

$$\Rightarrow \; cde(0.40)$$

Lista: $\{a(0.35),\ b(0.25),\ cde(0.40)\}$.

3ª fusione: i due più piccoli sono $b(0.25)$ e $a(0.35)$

$$\Rightarrow \; ab(0.60)$$

Lista: $\{ab(0.60),\ cde(0.40)\}$.

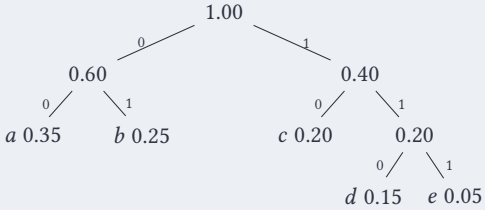
4ª fusione: restano solo $ab(0.60)$ e $cde(0.40)$

$$\Rightarrow \; \text{radice } (1.00)$$

Quattro fusioni per cinque simboli. In generale: $K - 1$ fusioni per K simboli.

L'albero e il codice

Leggendo dalla radice verso le foglie (0 a sinistra, 1 a destra):



s_i	a	b	c	d	e
$c(s_i)$	00	01	10	110	111

$$\bar{\ell}_{\text{Huff}} = 2 \cdot (0.35 + 0.25 + 0.20) + 3 \cdot (0.15 + 0.05) = 2.20 \text{ bit}$$

Huffman vs Shannon–Fano

Su questa sorgente i due algoritmi producono il medesimo codice:

$$\bar{\ell}_{\text{Huff}} = \bar{\ell}_{\text{SF}} = 2.20 \text{ bit}$$

Non è sempre così

Coincidenza dovuta alla struttura particolare di questa distribuzione. In generale:

$$\bar{\ell}_{\text{Huff}} \leq \bar{\ell}_{\text{SF}}$$

e la disuguaglianza può essere stretta.

Shannon–Fano può essere fortunato o sfortunato: Huffman è sempre ottimo, per costruzione.

Un caso sfavorevole per Shannon–Fano

Esempio

Variante con probabilità simili ma meno regolari:

$$p(A) = 0.35,\ \ p(B) = 0.17,\ \ p(C) = 0.17,\ \ p(D) = 0.16,\ \ p(E) = 0.15$$

Entropia: $H(X) \simeq 2.23$ bit.

La divisione più bilanciata al primo livello è $\{A, B\}$ vs $\{C, D, E\}$ (diff 0.04). Shannon–Fano la sceglie: A e B finiscono entrambi con 2 bit.

s_i	A	B	C	D	E
$c(s_i)$	00	01	10	110	111

$$\bar{\ell}_{\mathrm{SF}} = 0.35 \cdot 2 + 0.17 \cdot 2 + 0.17 \cdot 2 + 0.16 \cdot 3 + 0.15 \cdot 3 = 2.31 \text{ bit}$$

Huffman trova la parola da 1 bit

Huffman accorpa $E + D$, poi $B + C$, poi $[DE] + [BC]$, infine $A + [BCDE]$: A risulta figlio diretto della radice, con parola da 1 bit.

s_i	A	B	C	D	E
$c(s_i)$	0	110	111	100	101

$$\bar{\ell}_{\text{Huff}} = 0.35 \cdot 1 + 0.65 \cdot 3 = 2.30 \text{ bit}$$

Confronto

$\bar{\ell}_{\text{Huff}} = 2.30$ bit $< \bar{\ell}_{\text{SF}} = 2.31$ bit.

Shannon–Fano aveva cercato la *divisione bilanciata*; Huffman, costruendo dal basso, ha trovato la *struttura globale* ottimale.

Ottimalità di Huffman

Teorema (Huffman, 1952)

Tra tutti i codici prefisso su un dato alfabeto, quello prodotto dall'algoritmo di Huffman ha la minima lunghezza media.

Punto chiave dell'argomento

In qualunque codice prefisso ottimale, i due simboli con probabilità più bassa sono fratelli: differiscono soltanto nell'ultimo bit.

Quindi possiamo accorparli in un unico simbolo virtuale di probabilità $p_i + p_j$, riducendo il problema da K a $K-1$ simboli. Per induzione la procedura è ottimale.

Limiti strutturali e rimedi

Il bound $\bar{\ell}_{\text{Huff}} < H(X) + 1$ può essere quasi raggiunto quando esiste un simbolo molto probabile.

Il caso $p_{\max} > 1/2$

Il simbolo più frequente riceve il codice da 1 bit, ma il suo contenuto informativo è $-\log_2 p_{\max} < 1$ bit. L'overhead per ogni occorrenza vale

$$\Delta = 1 + \log_2 p_{\max}$$

e pesa sulla media in proporzione a $p_{\max}$.

Rimedio: codificare a blocchi

Raggruppare n simboli consecutivi come un'unica entità. La lunghezza media per simbolo scende sotto $H(X) + \frac{1}{n}$, al costo di una tabella di K^n voci.

Proprietà e conclusione

- È sempre un codice prefisso (costruito su un albero binario).
- Rispetta lo stesso bound di Shannon–Fano:

$$H(X) \leq \bar{\ell}_{\text{Huff}} < H(X) + 1$$

- Ma in più: è ottimale tra i codici prefisso.
- Per avvicinarsi ulteriormente a $H(X)$ si codificano blocchi di simboli (k-gruppi), spalmando l'arrotondamento.

In pratica

Huffman è il cuore di moltissimi formati di compressione: `ZIP`, `gzip`, `JPEG`, `MP3`, `PNG`, ... quasi sempre c'è un livello di codifica di Huffman.

Codifica di canale

Fin qui: il canale era perfetto

Tutta la discussione sulla compressione ha assunto un'ipotesi comoda:

Bruno riceve *esattamente* ciò che Anna ha inviato.

In questo scenario l'obiettivo è unico: ridurre la lunghezza del messaggio trasmesso, sfruttando la struttura statistica della sorgente.

Domanda

E se il canale non fosse perfetto?

Canali rumorosi

In molti scenari reali i simboli trasmessi possono arrivare modificati a destinazione:

- trasmissione radio o satellitare con interferenze
- copia di dati su supporti fisici (USB, hard disk, DVD)
- trascrizione biologica del DNA in RNA

Il "canale" è un concetto generale

Ogni processo che trasforma un messaggio in ingresso in un messaggio in uscita, con possibilità di introdurre errori, è un canale rumoroso.

Non basta più chiedersi quanto sia *corta* la rappresentazione: bisogna chiedersi quanto sia affidabile la trasmissione.

Come si descrive un canale rumoroso

La descrizione del canale non è più deterministica, ma probabilistica.

Nel caso più semplice (alfabeto binario $\{0, 1\}$), il canale è descritto da una sola grandezza: la probabilità f che un singolo bit venga *invertito* durante la trasmissione.

Canale binario simmetrico

Con probabilità $1 - f$ il bit arriva corretto; con probabilità f viene invertito.
Il parametro f è lo stesso per $0 \to 1$ e per $1 \to 0$: da qui il nome "simmetrico".

- f piccolo $\Rightarrow$ canale poco rumoroso
- $f \approx 0.5 \Rightarrow$ canale quasi inutile: l'uscita è praticamente indipendente dall'ingresso

La ridondanza come risorsa

Come affrontare il rumore? L'idea chiave è introdurre ridondanza.

Apparente paradosso

La compressione *elimina* ridondanza. La codifica di canale la *introduce*. Sembrano in contraddizione.

In realtà non lo sono: la ridondanza aggiunta dalla codifica di canale è strutturata e controllata. Fornisce vincoli aggiuntivi che permettono di capire quali bit siano stati alterati.

Esempio

Se so che la parola trasmessa era una di sole quattro possibili, e ricevo qualcosa di diverso da tutte e quattro, capisco che c'è stato un errore — e posso scegliere la più vicina.

Il codice a ripetizione

Il metodo più semplice: trasmettere ogni bit più volte.

Ripetizione 3

Invece di trasmettere 0, trasmetto 000. Invece di 1, trasmetto 111.
Bruno riceve tre copie e sceglie il valore che compare almeno due volte: *voto di maggioranza.*

Esempio

Anna vuole trasmettere $s = 0\,0\,1\,0\,1\,1\,0$.
Trasmette effettivamente: 000 000 111 000 111 111 000.
Se il canale altera al più un bit per terzetto, la maggioranza resta corretta.

Il codice a ripetizione corregge un errore per terzetto; fallisce se gli errori in una stessa tripla sono due o tre.

Quanto migliora la situazione?

Con un canale che sbaglia ogni bit con probabilità $f = 0.1$, la probabilità che un terzetto venga decodificato erroneamente è:

$$\underbrace{3f^2(1-f)}_{\text{esattamente 2 errori}} + \underbrace{f^3}_{\text{3 errori}} \approx 0.028$$

Si scende da 10% a 2.8%: un miglioramento reale.

Il costo

Per ogni bit informativo se ne trasmettono tre. La lunghezza del messaggio è triplicata.

Il vero problema

Per raggiungere probabilità di errore dell'ordine di 10^{-15} (necessaria, ad esempio, per copiare un terabyte senza errori) servirebbero 68 ripetizioni: del tutto impraticabile.

Servono codici più intelligenti

Il codice a ripetizione è intuitivo ma inefficiente: la ridondanza cresce in modo incontrollato.

Domanda

È possibile correggere errori con molta meno ridondanza?

Risposta

Sì. Invece di proteggere ogni singolo bit ripetendolo, si proteggono blocchi di bit introducendo ridondanza in modo strutturato. È l'idea alla base dei codici di Hamming.

La distanza di Hamming

Introduciamo uno strumento fondamentale per misurare quanto due parole binarie differiscono.

Distanza di Hamming $\delta(x, y)$

Dati due messaggi binari x e y della stessa lunghezza, la loro distanza di Hamming è il numero di posizioni in cui differiscono.

Esempio

$\delta(0111, 0010) = 2$ (differiscono nel 2° e 4° bit)

$\delta(1100, 0000) = 2$ (differiscono nei primi due bit)

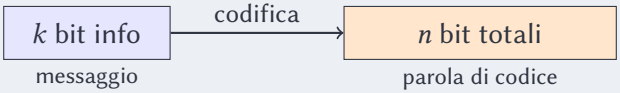
La distanza di Hamming

Il significato fisico

Un errore singolo sul canale trasla la parola trasmessa di *distanza 1*. Due errori di distanza 2. E così via. La distanza di Hamming è esattamente la misura del "numero di errori" occorsi.

Codici a blocchi (n, k): l'idea generale

Un codice a blocchi (n, k) prende un messaggio di k bit e lo trasforma in una parola di codice di $n > k$ bit, aggiungendo $n - k$ bit di ridondanza.



Non tutte le 2^n sequenze binarie di lunghezza n sono parole di codice valide: solo 2^k di esse lo sono.

Il tasso di codice

Il rapporto $R = k/n$ misura la frazione di bit informativi: più è vicino a 1, più il codice è efficiente. Aggiungere ridondanza significa ridurre R.

Esempio minimo: il codice $(2, 1)$

Il codice $(2, 1)$ è il caso più semplice: 1 bit informativo, 1 bit di ridondanza.

Costruzione

Il bit di ridondanza t_2 è semplicemente una copia del bit informativo s_1:

$$t_1 = s_1, \qquad t_2 = s_1$$

Le parole di codice valide sono solo due: `00` e `11`.

Esempio

Se Bruno riceve `01` o `10`, sa che c'è stato un errore: nessuna delle due è una parola valida.
Ma non sa quale delle due parole valide (`00` o `11`) fosse quella originale.

Il codice $(2, 1)$ rileva un errore singolo ma non lo corregge. La distanza minima è $d_{\min} = 2$.

Esempio: il codice $(3, 2)$

Il codice $(3, 2)$ codifica 2 bit informativi in 3, aggiungendo 1 bit di parità.

Costruzione

Il bit di parità t_3 è la somma modulo 2 dei due bit informativi:

$$t_3 = s_1 \oplus s_2$$

($t_3 = 0$ se s_1 e s_2 sono uguali; $t_3 = 1$ se sono diversi)

Esempio: il codice $(3, 2)$

$s_1 s_2$	parola $t_1 t_2 t_3$
00	0 0 0
01	0 1 1
10	1 0 1
11	1 1 0

Qualunque coppia di parole valide differisce in esattamente 2 posizioni: $d_{\min} = 2$.
Errori singoli rilevabili, ma non correggibili.

Perché $(3, 2)$ non corregge gli errori

Esempio

Supponiamo che Anna trasmetta 000 e Bruno riceva 001.

Bruno sa che 001 non è una parola valida (la parità non torna). Sa che c'è stato un errore.

Domanda

Quale parola valida era quella originale?

$$\delta(001, 000) = 1 \quad \leftarrow \text{forse era questa?}$$
$$\delta(001, 011) = 1 \quad \leftarrow \text{o forse questa?}$$

Perché $(3, 2)$ non corregge gli errori

Risposta

001 è a distanza 1 da *due* parole valide diverse: non c'è una scelta univoca. Con $d_{\min} = 2$ le parole valide sono troppo vicine tra loro per permettere la correzione.

Per correggere servono parole valide che distino almeno 3 tra loro: $d_{\min} \geq 3$.

Rilevare e correggere: la regola generale

La distanza minima tra le parole valide determina le capacità del codice:

Regola generale

Un codice con distanza minima $d_{\min}$ può:

- rilevare fino a $d_{\min} - 1$ errori
- correggere fino a $\lfloor (d_{\min} - 1)/2 \rfloor$ errori

$d_{\min}$	rileva	corregge
2	1 errore	0 errori
3	2 errori	1 errore
5	4 errori	2 errori

Rilevare e correggere: la regola generale

Intuizione

Per correggere 1 errore, ogni parola corrotta deve essere più vicina alla parola originale che a qualunque altra parola valida. Servono “sfere” di raggio 1 attorno a ciascuna parola valida che non si sovrappongano.

L'idea del bit di parità

Come costruire codici con $d_{\min} \geq 3$ senza sprecare troppa ridondanza?

Bit di parità

Un bit di parità è un bit aggiunto al messaggio tale che la somma modulo 2 di un certo sottoinsieme di bit (informativi + parità) valga sempre 0.

È come un testimone: osserva un gruppo di bit e dichiara "in questo gruppo c'è un numero pari di uno".

L'idea del bit di parità

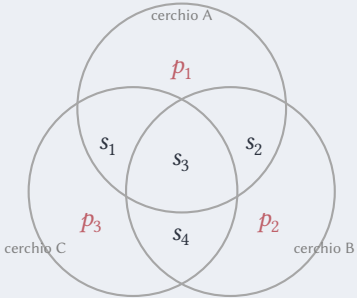
Cosa succede con un errore

Se un bit del gruppo controllato viene invertito, il numero di uno diventa dispari: la dichiarazione del testimone è contraddetta. L'errore viene rilevato.

Con un *solo* bit di parità sappiamo che c'è un errore, ma non *dove*. Usando più testimoni che controllano sottoinsiemi sovrapposti, il pattern di contraddizioni punta al bit sbagliato.

Il codice $(7, 4)$: il diagramma dei tre cerchi

Nel codice di Hamming $(7, 4)$ i 7 bit vengono disposti nelle 7 regioni di tre cerchi sovrapposti.



I **4 bit informativi** s_1, s_2, s_3, s_4 occupano le regioni centrali (intersezioni di due o tre cerchi).

I **3 bit di parità** p_1, p_2, p_3 occupano le regioni esterne (ciascuno in un solo cerchio).

Ogni cerchio controlla un gruppo di bit:

- **A**: s_1, s_2, s_3, p_1
- **B**: s_2, s_3, s_4, p_2
- **C**: s_1, s_3, s_4, p_3

Come si costruiscono i bit di parità

Ogni bit di parità viene scelto in modo che la somma modulo 2 di tutti i bit nel suo cerchio sia zero.

Le tre equazioni di parità

$$p_1 = s_1 \oplus s_2 \oplus s_3$$

$$p_2 = s_2 \oplus s_3 \oplus s_4$$

$$p_3 = s_1 \oplus s_3 \oplus s_4$$

Come si costruiscono i bit di parità

Esempio

Vogliamo trasmettere $s_1 s_2 s_3 s_4 = 1\,0\,1\,1$. I bit di parità sono:

$$p_1 = 1 \oplus 0 \oplus 1 = 0, \quad p_2 = 0 \oplus 1 \oplus 1 = 0, \quad p_3 = 1 \oplus 1 \oplus 1 = 1$$

La parola trasmessa è $s_1 s_2 s_3 s_4 p_1 p_2 p_3 = 1\,0\,1\,1\;\;0\,0\,1$.

Qualunque coppia di parole valide differisce in almeno 3 posizioni: $d_{\min} = 3$, quindi il codice corregge un errore singolo.

La sindrome: leggere il pattern di violazioni

Bruno riceve 7 bit e ricalcola le tre somme di parità. Chiama il risultato sindrome.

Se nessun errore è occorso, tutte e tre le somme valgono 0: sindrome $= (0, 0, 0)$.

Se invece c'è un errore nel bit b, solo i cerchi che *contengono* b risultano violati.

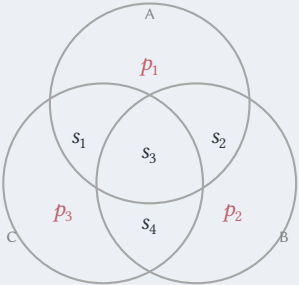
La sindrome: leggere il pattern di violazioni

bit errato	cerchio A	cerchio B	cerchio C	sindrome
nessuno	0	0	0	$(0, 0, 0)$ — ok
s_1	1	0	1	$(1, 0, 1)$
s_2	1	1	0	$(1, 1, 0)$
s_3	1	1	1	$(1, 1, 1)$
s_4	0	1	1	$(0, 1, 1)$
p_1	1	0	0	$(1, 0, 0)$
p_2	0	1	0	$(0, 1, 0)$
p_3	0	0	1	$(0, 0, 1)$

Ogni bit ha una firma unica: 8 sindromi distinte per 8 casi (nessun errore + 7 bit in errore).

Perché ogni bit ha una firma unica?

Il segreto sta nella scelta di *quali* bit appartengono a *quali* cerchi.



Ogni bit è in un sottoinsieme *diverso* di cerchi:

bit	A	B	C
p_1	•		
p_2		•	
p_3			•
s_1	•		•
s_2	•	•	
s_3	•	•	•
s_4		•	•

Nessuna riga è uguale a un'altra: ogni bit ha la propria "identità" nei cerchi. La sindrome è una firma univoca.

Esempio completo: dalla sorgente alla trasmissione

Esempio

Anna vuole trasmettere il messaggio $s_1 s_2 s_3 s_4 = 1101$.

Calcolo dei bit di parità:

$$p_1 = s_1 \oplus s_2 \oplus s_3 = 1 \oplus 1 \oplus 0 = 0$$
$$p_2 = s_2 \oplus s_3 \oplus s_4 = 1 \oplus 0 \oplus 1 = 0$$
$$p_3 = s_1 \oplus s_3 \oplus s_4 = 1 \oplus 0 \oplus 1 = 0$$

Parola trasmessa: 1101 000

Verifica

Ogni cerchio: A: $1 \oplus 1 \oplus 0 \oplus 0 = 0$ B: $1 \oplus 0 \oplus 1 \oplus 0 = 0$ C: $1 \oplus 0 \oplus 1 \oplus 0 = 0$

Esempio completo: ricezione e correzione

Esempio

Parola trasmessa: 1101 000
Il canale inverte il terzo bit: Bruno riceve $r = 1111\ 000$.

Bruno ricalcola le tre somme di parità:

$$\begin{aligned}
\text{A} &: r_{s_1} \oplus r_{s_2} \oplus r_{s_3} \oplus r_{p_1} = 1 \oplus 1 \oplus 1 \oplus 0 = 1 \quad \text{(violata!)} \\
\text{B} &: r_{s_2} \oplus r_{s_3} \oplus r_{s_4} \oplus r_{p_2} = 1 \oplus 1 \oplus 1 \oplus 0 = 1 \quad \text{(violata!)} \\
\text{C} &: r_{s_1} \oplus r_{s_3} \oplus r_{s_4} \oplus r_{p_3} = 1 \oplus 1 \oplus 1 \oplus 0 = 1 \quad \text{(violata!)}
\end{aligned}$$

Sindrome: $(1, 1, 1) \Rightarrow$ dalla tabella, il bit errato è s_3 (terzo bit).

Correzione: Bruno inverte il terzo bit: $1101\ 000$ e legge $s_1 s_2 s_3 s_4 = 1101$.

Quanto è efficiente il codice $(7, 4)$?

Confrontiamo le strategie per correggere un errore singolo:

	Ripetizione 3	Hamming $(3, 2)$	Hamming $(7, 4)$
Bit informativi (k)	1	2	4
Bit totali (n)	3	3	7
Bit di ridondanza	2	1	3
Tasso $R = k/n$	$1/3$	$2/3$	$4/7$
Correzione	1 errore	solo rileva	1 errore

Conclusione

Hamming $(7, 4)$ corregge errori singoli usando solo 3 bit di ridondanza per 4 bit informativi. Il 57% dei bit porta informazione reale, contro il 33% della ripetizione semplice.

In generale, con r bit di parità si possono proteggere fino a $2^r - r - 1$ bit informativi: la ridondanza cresce molto più lentamente dei bit protetti.

Codifica di sorgente vs codifica di canale

Ricapitoliamo il quadro complessivo:

Codifica di sorgente (compressione)

Obiettivo: ridurre la lunghezza del messaggio.
Strumento: eliminare ridondanza sfruttando la struttura statistica.
Esempi: Huffman, Shannon–Fano.

Codifica di canale (correzione di errori)

Obiettivo: rendere la trasmissione affidabile su un canale rumoroso.
Strumento: aggiungere ridondanza strutturata.
Esempi: codici a ripetizione, codici di Hamming.

In un sistema di comunicazione reale le due codifiche si applicano in sequenza: prima si comprime, poi si aggiunge ridondanza per il canale.